

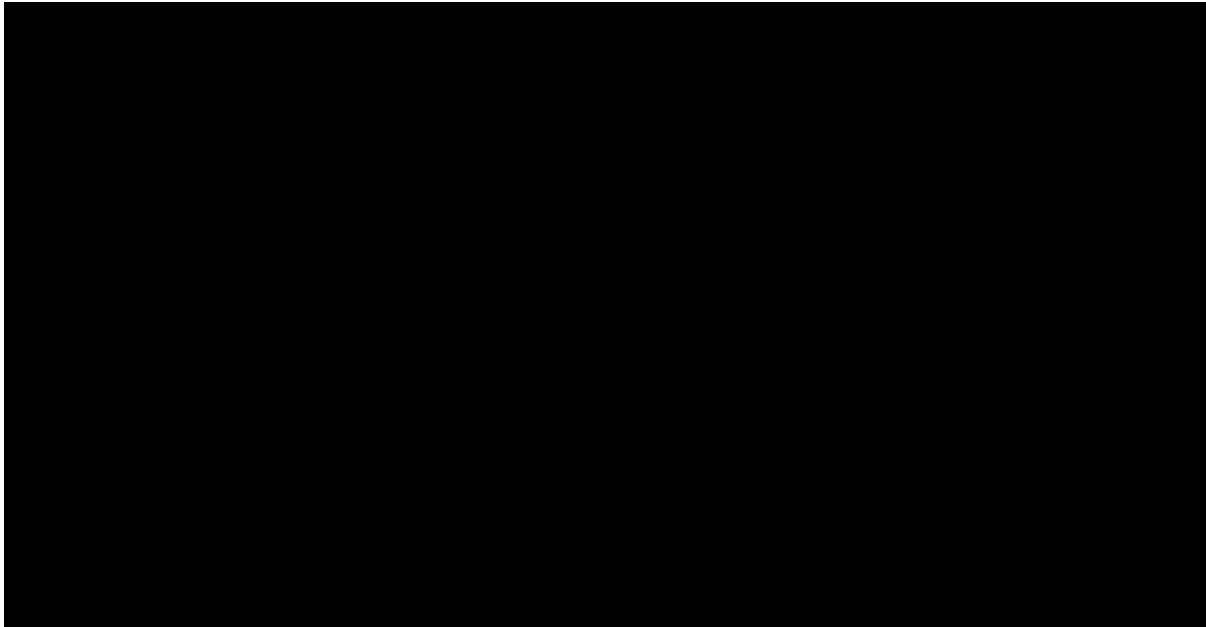
Computer Vision

CSC-764

Lecture 1

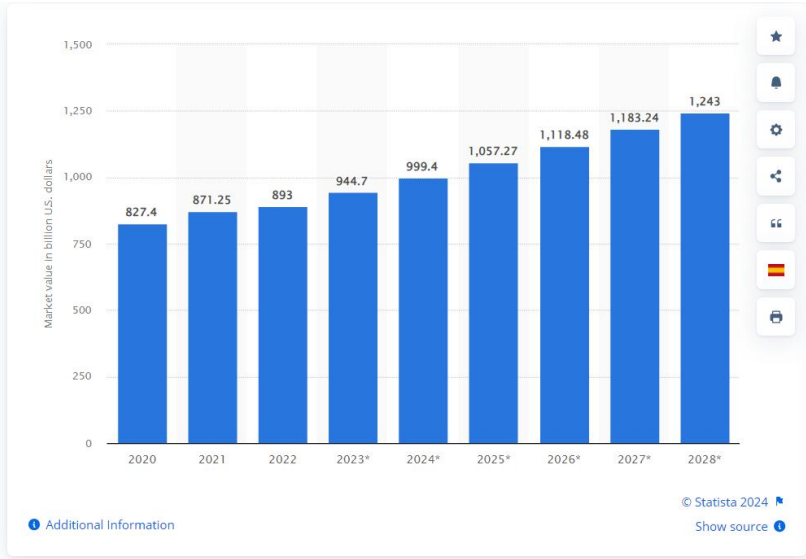
Dr. Shahid Ismail

Shahid Ismail, PostDoctorate



Shahid Ismail, PostDoctorate

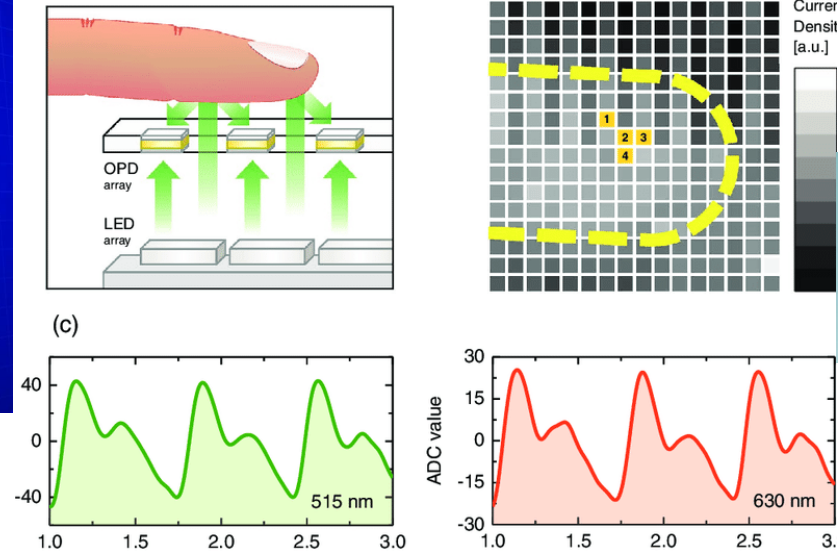
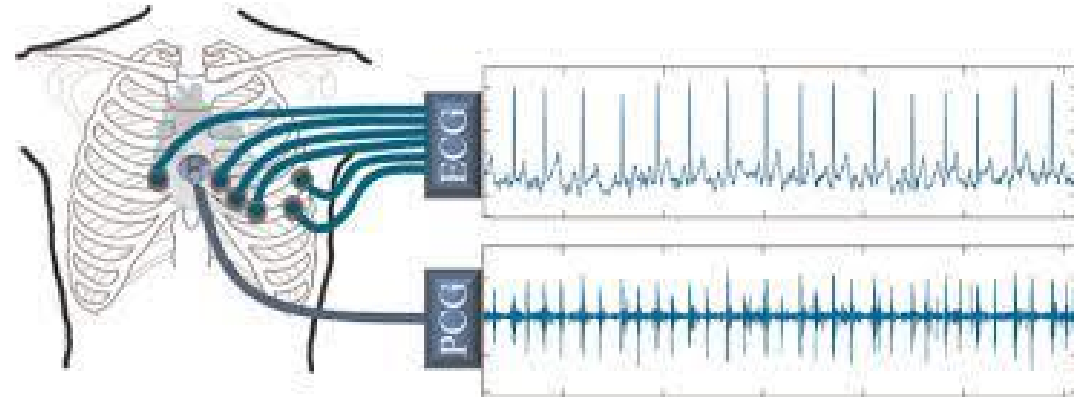
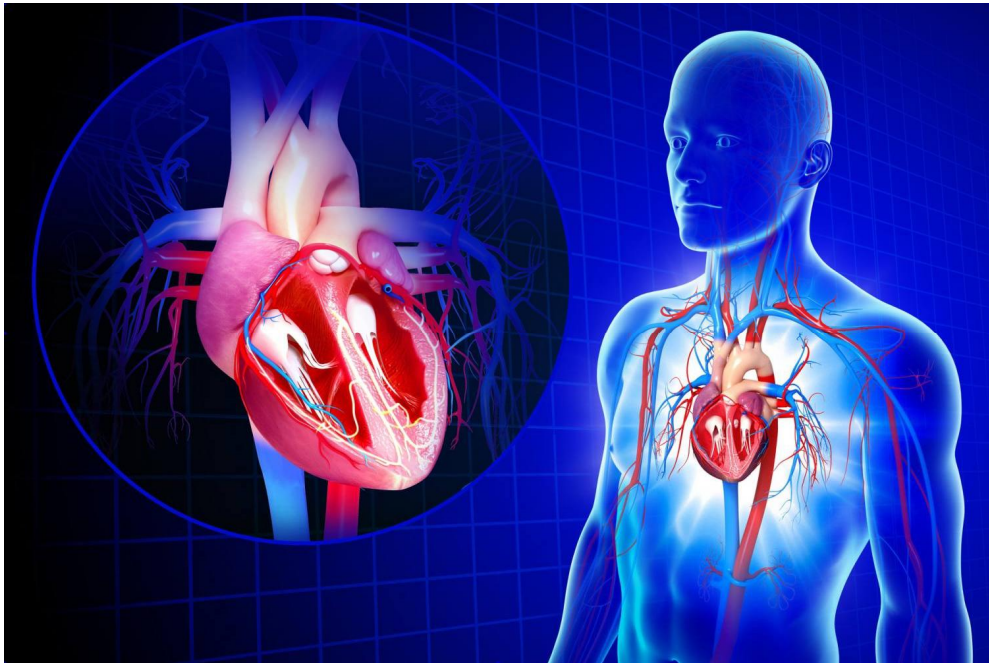
Global Dairy Top 20 Industry Leaders, 2023



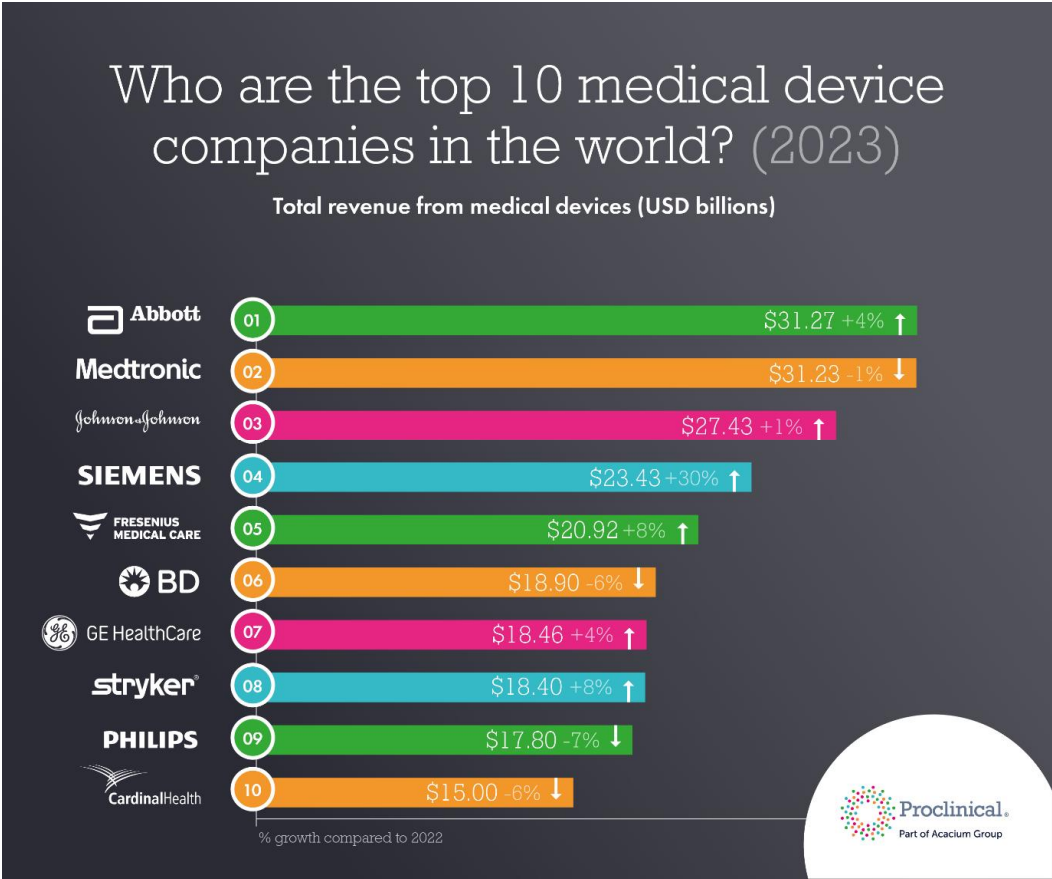
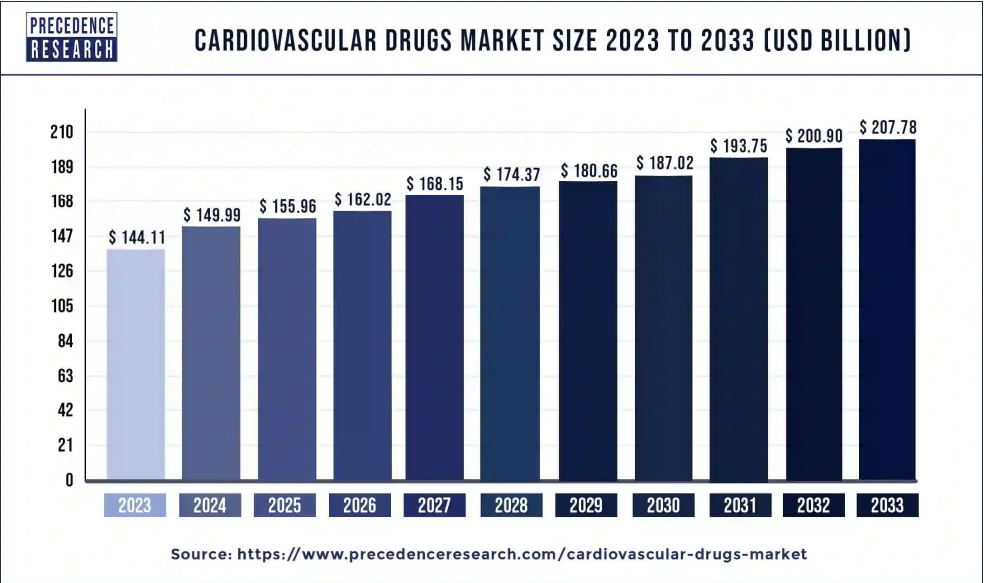
Rank 2023	Rank 2022	Company	Country of origin and main operation countries	Dairy turnover 2022 1 Billion US\$	Dairy turnover 2022 1 Billion Euros
1	1	Lactalis	France	28.6 ^e	27.2 ^e
2	4	Dairy Farmers of America	United States of America	24.5	23.3
3	2	Nestlé	Switzerland	23.3 ^e	22.1 ^e
4	3	Danone	France	21.2 ^e	21.1 ^e
5	5	Yili	China	18.3 ^e	17.3 ^e
6	9	Arla Foods	Denmark/Sweeden	14.5	13.8
7	8	FrieslandCampina	Netherlands	14.4 ^e	13.7 ^e
8	7	Mengniu	China	14.4 ^e	13.7 ^e
9	6	Fonterra	New Zealand	14.2 ^e	13.4 ^e
10	10	Saputo	Canada	13.7	13.0

Our budget: 67.84 Billion \$

Dr. Shahid Ismail

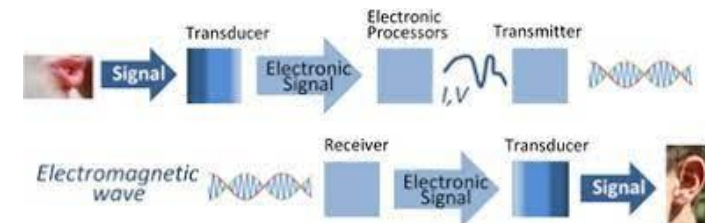
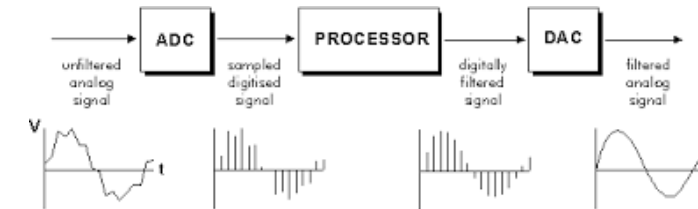
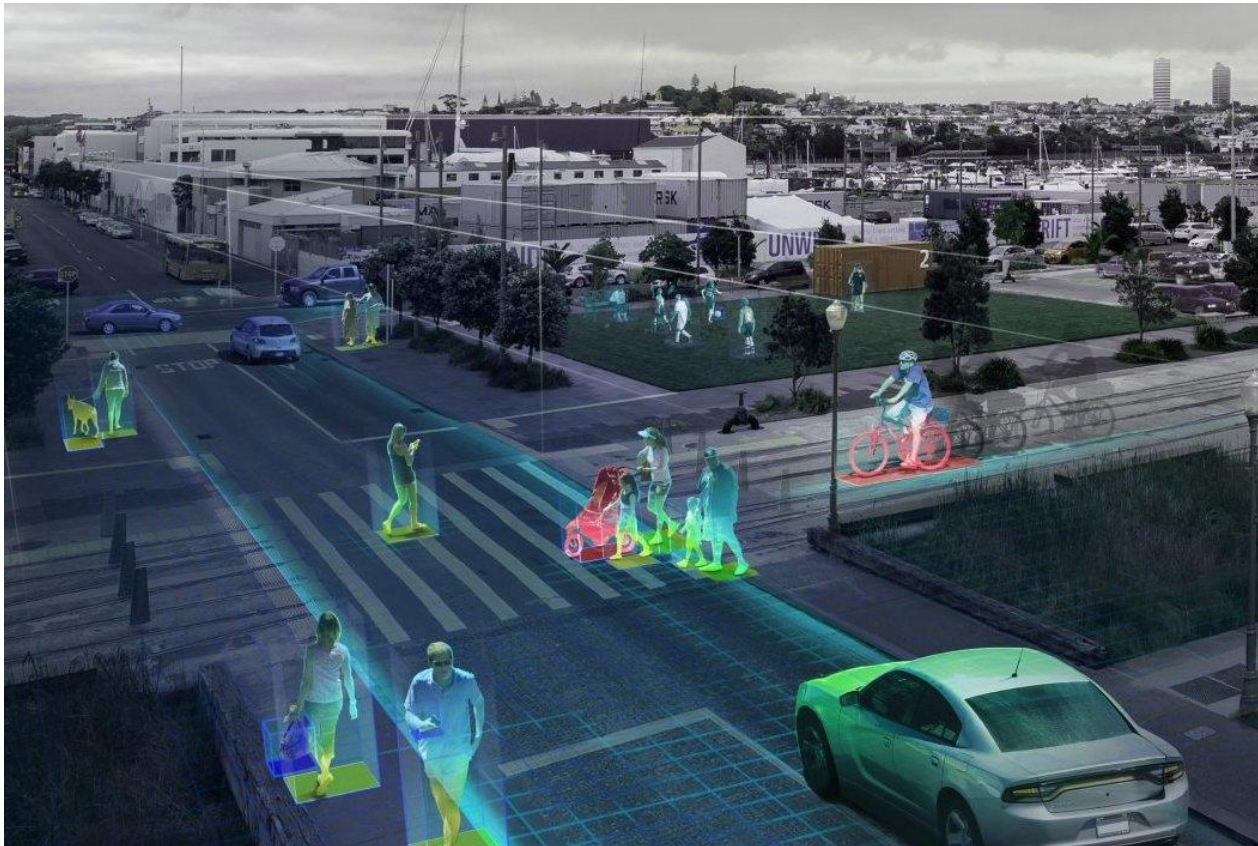


Dr. Shahid Ismail

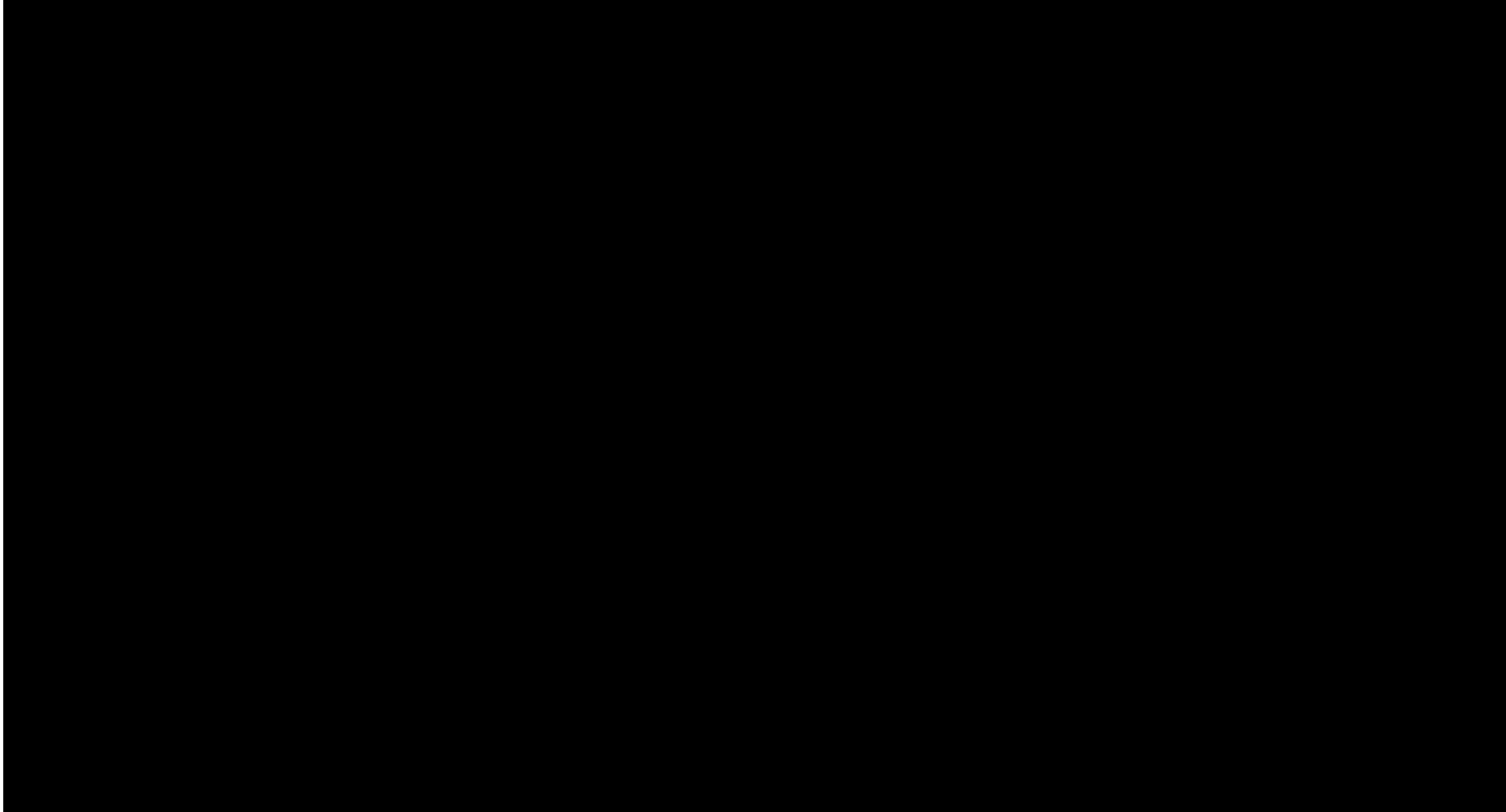


Our budget: 67.84 Billion U.S.

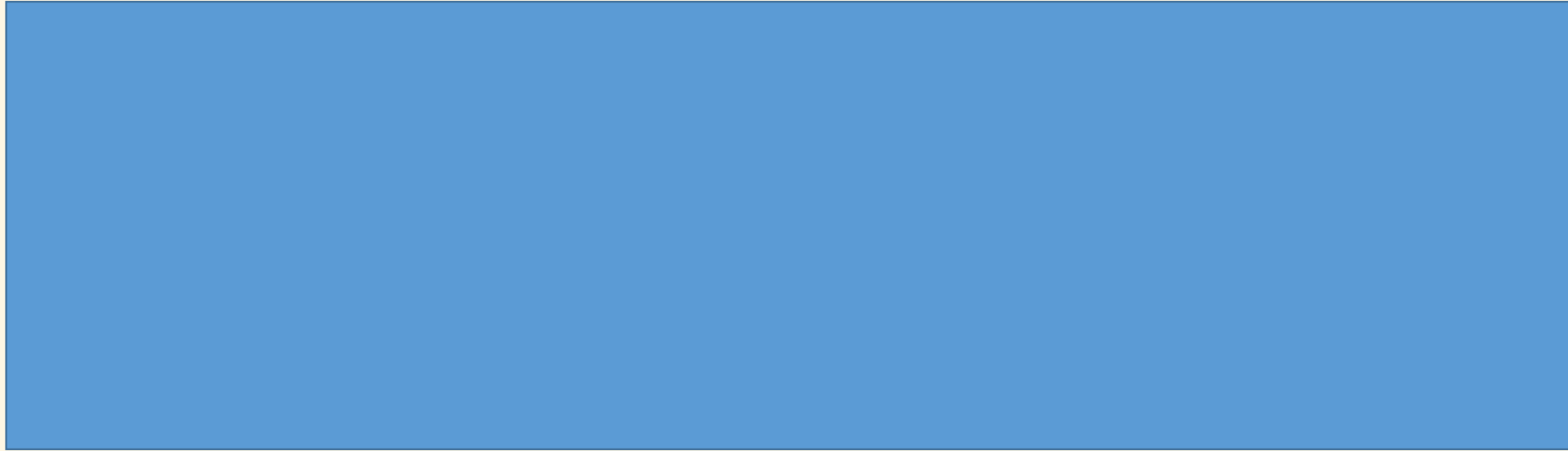
Eng. Shahid Ismail



Motivation



How Does Computer Vision Work?



Human Vision



Input



Eye



Brain

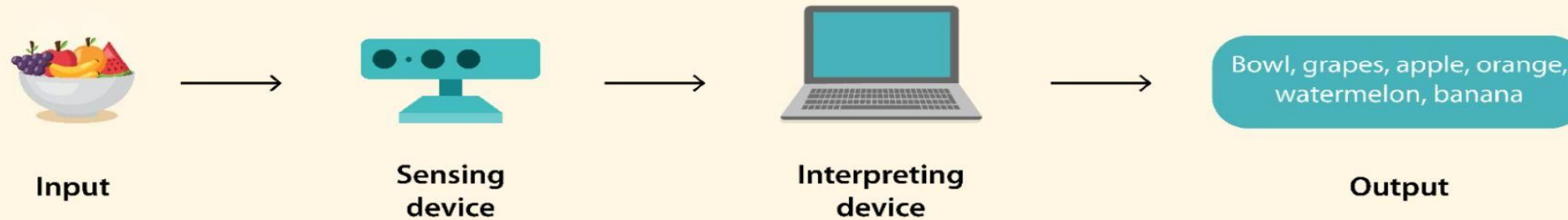


Bowl, grapes, apple, orange,
watermelon, banana

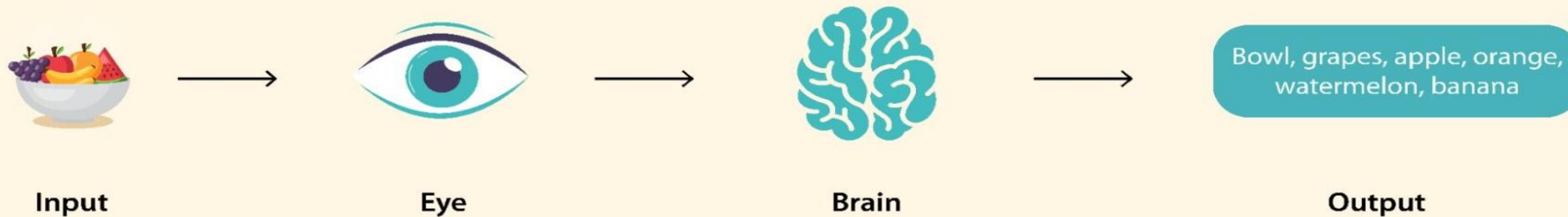
Output

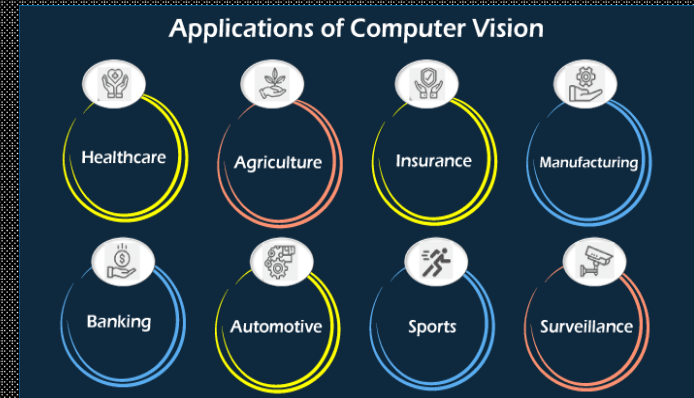
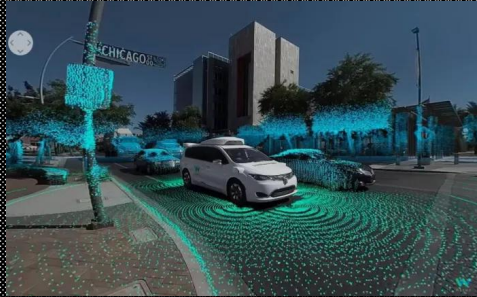
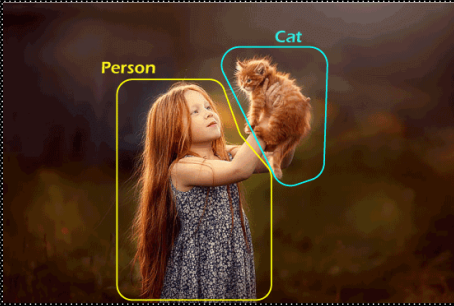
How Does Computer Vision Work?

Computer Vision



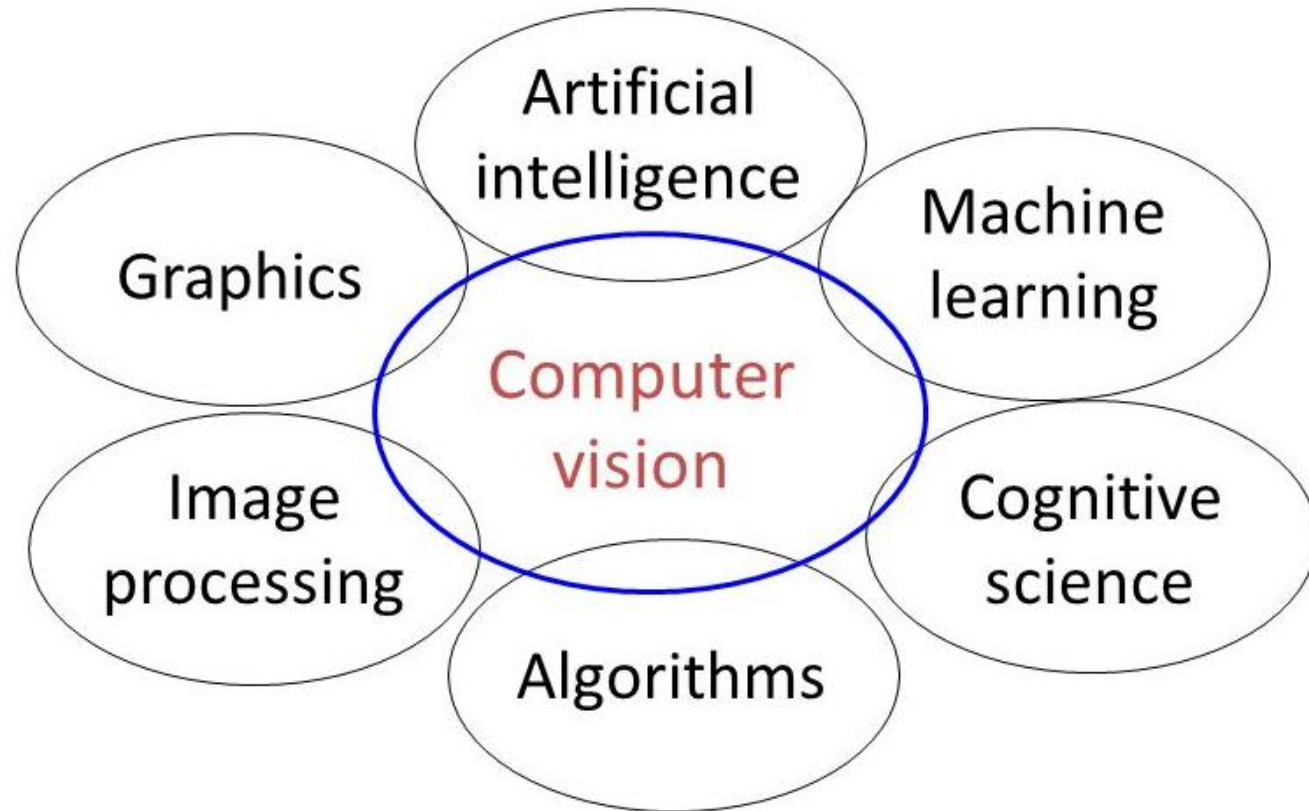
Human Vision



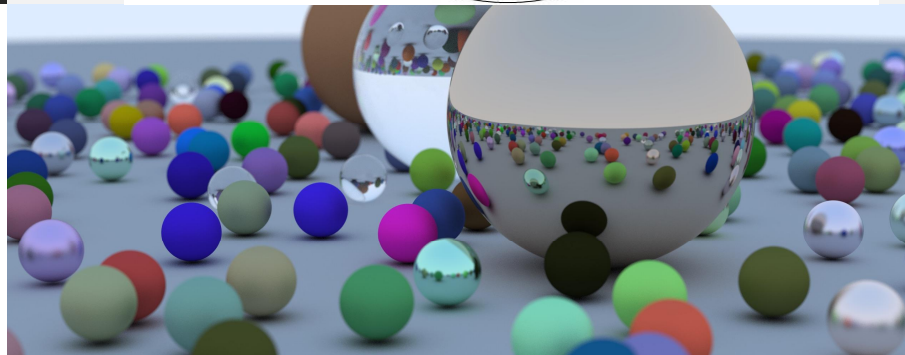
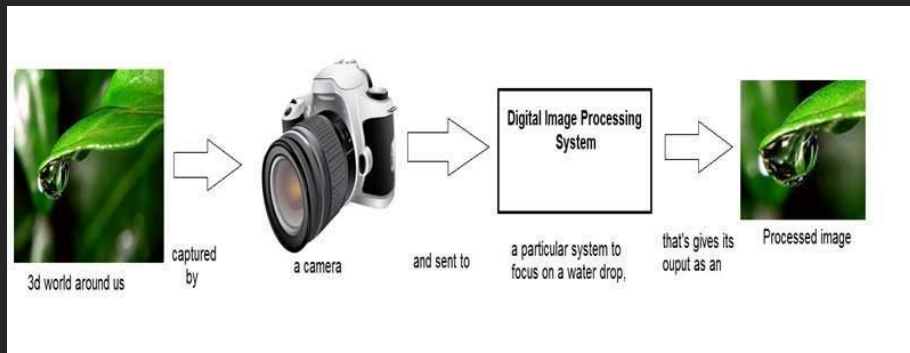
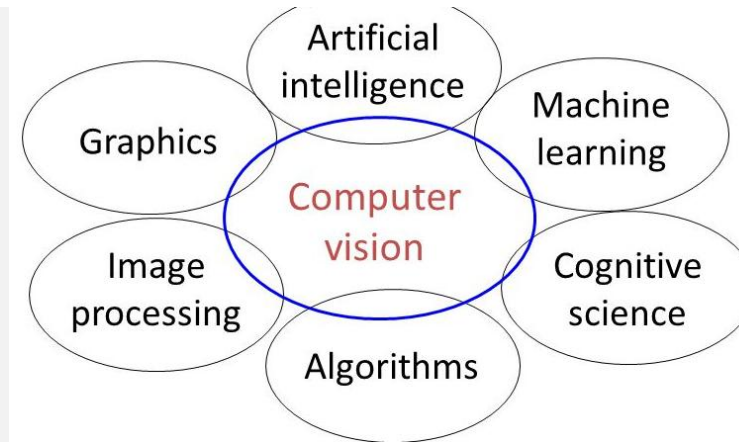


Applications

Related Disciplines



Related Disciplines



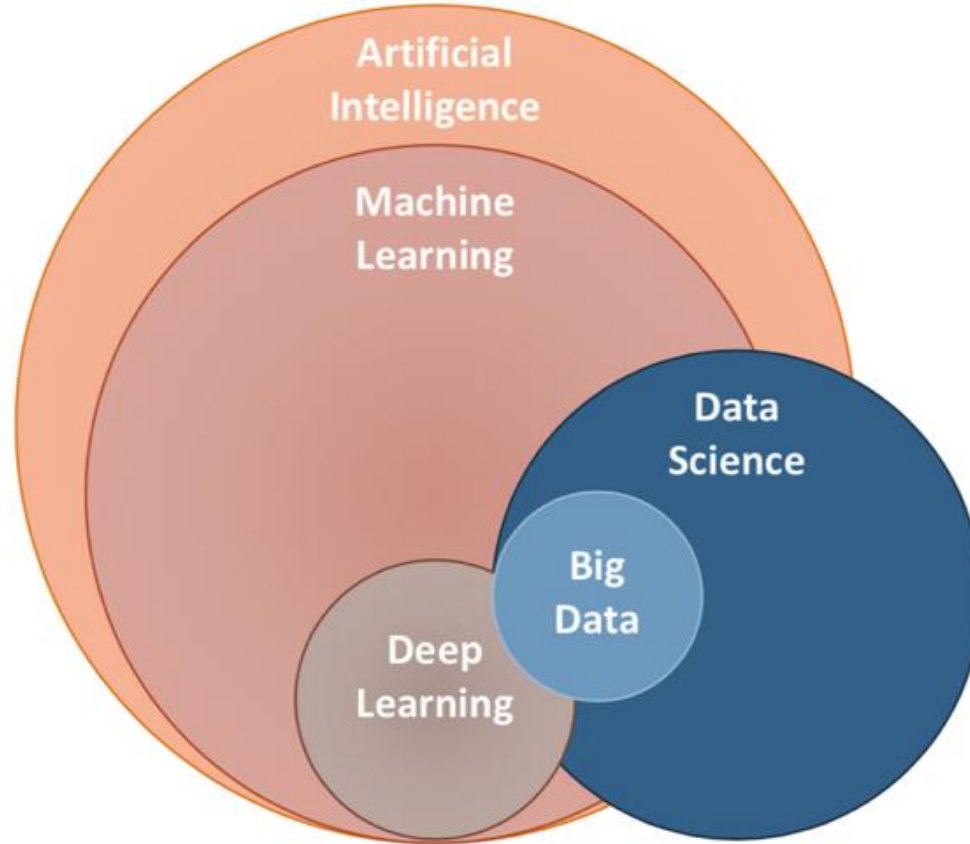
```

20 unsigned int len1 = s1.size();
21 const size_t len2 = col[len2+1], prevCol;
22 vector<unsigned int> col(len2+1);
23 for (unsigned int i = 0; i < prevCol.size(); i++)
24     prevCol[i] = i;
25 for (unsigned int i = 0; i < len1; i++) {
26     col[0] = i+1;
27     for (unsigned int j = 0; j < len2; j++)
28         col[j+1] = std::min( std::min(prevCol[i+j] + j) + 1, col[j] +
29                             prevCol[j] + (s1[i]==s2[j]) ? 0 : 1 );
30     col.swap(prevCol);

```



AI & ML



Objectives



By the end of this course, the students would have developed an understanding of the problems in simulating human perception into machines.



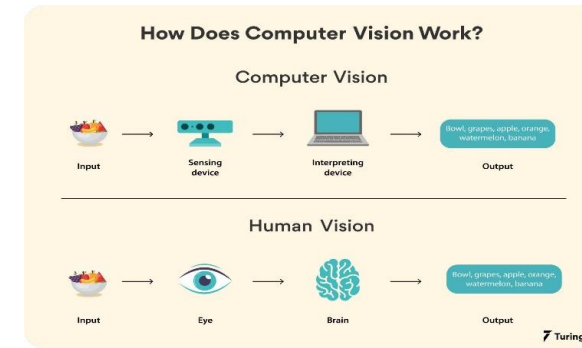
Students will have a thorough understanding of the state of the art computer vision methods, algorithms and results.



The students will also be able to apply the tools and techniques learned to solve practical vision related problems.

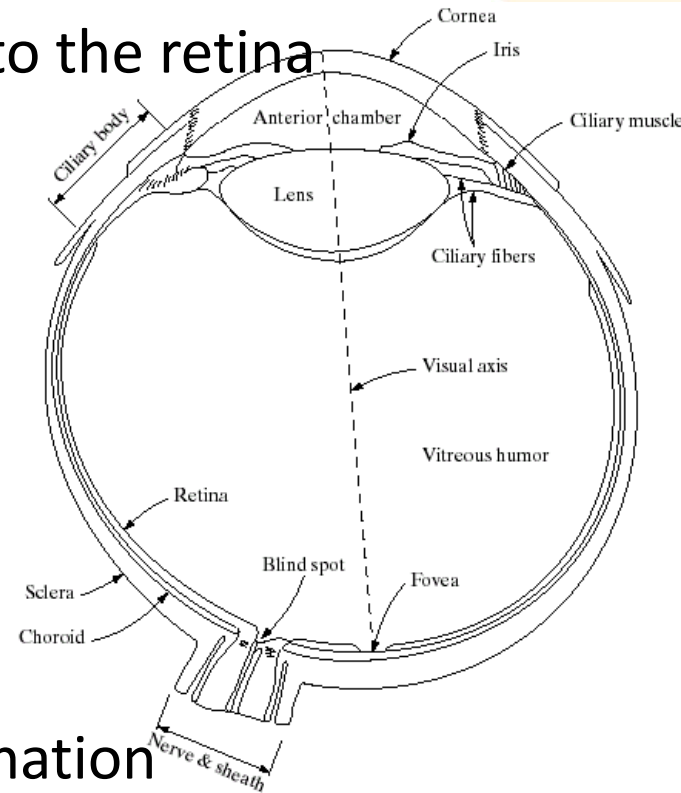
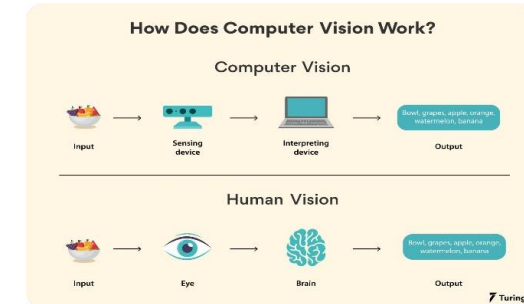
Human Visual System

- The best vision model we have!
- Knowledge of how images form in the eye can help us with processing digital images
- We will take just a whirlwind tour of the human visual system

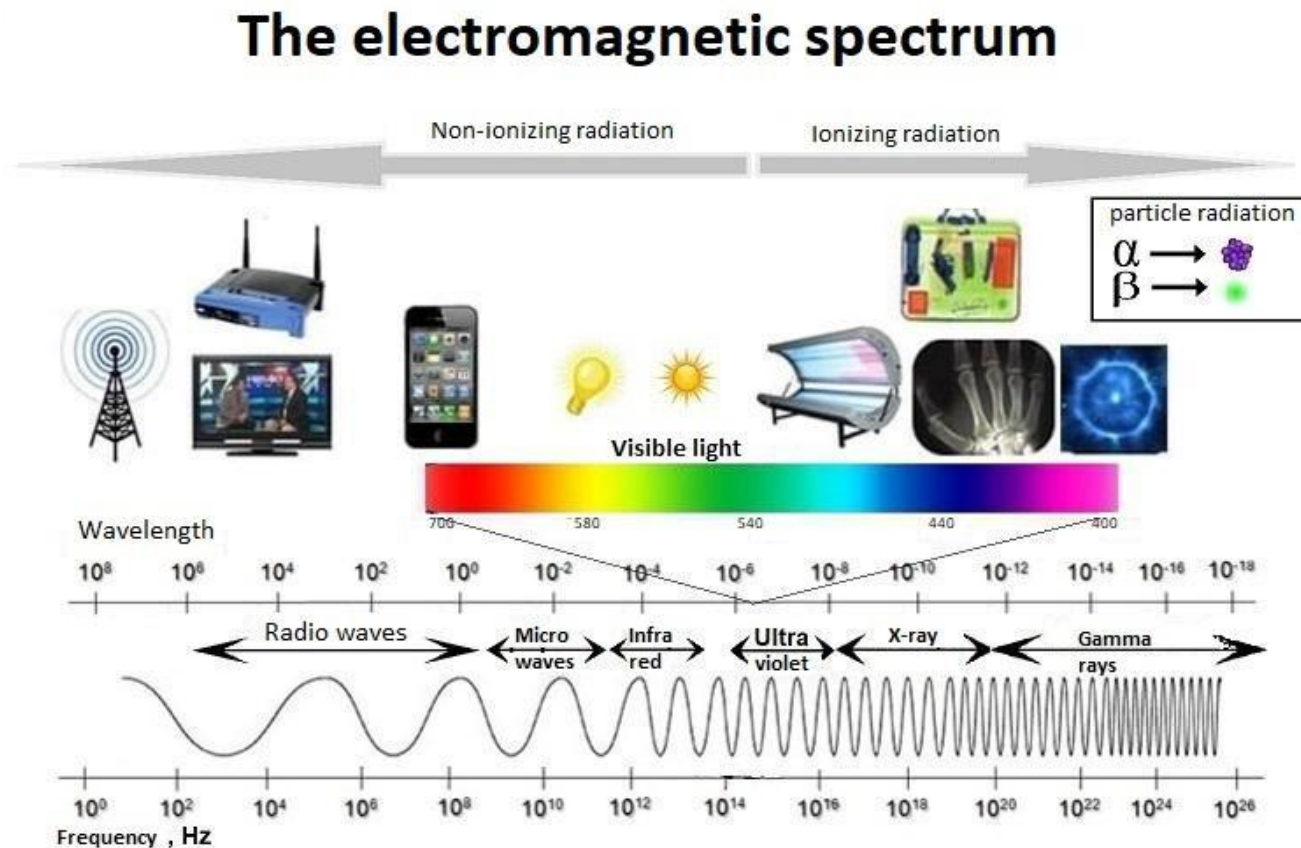


Structure Of The Human Eye

- The lens focuses light from objects onto the retina
- The retina is covered with light receptors called *cones* (6-7 million) and *rods* (75-150 million)
- Cones are concentrated around the fovea and are very sensitive to colour
- Rods are more spread out and are sensitive to low levels of illumination

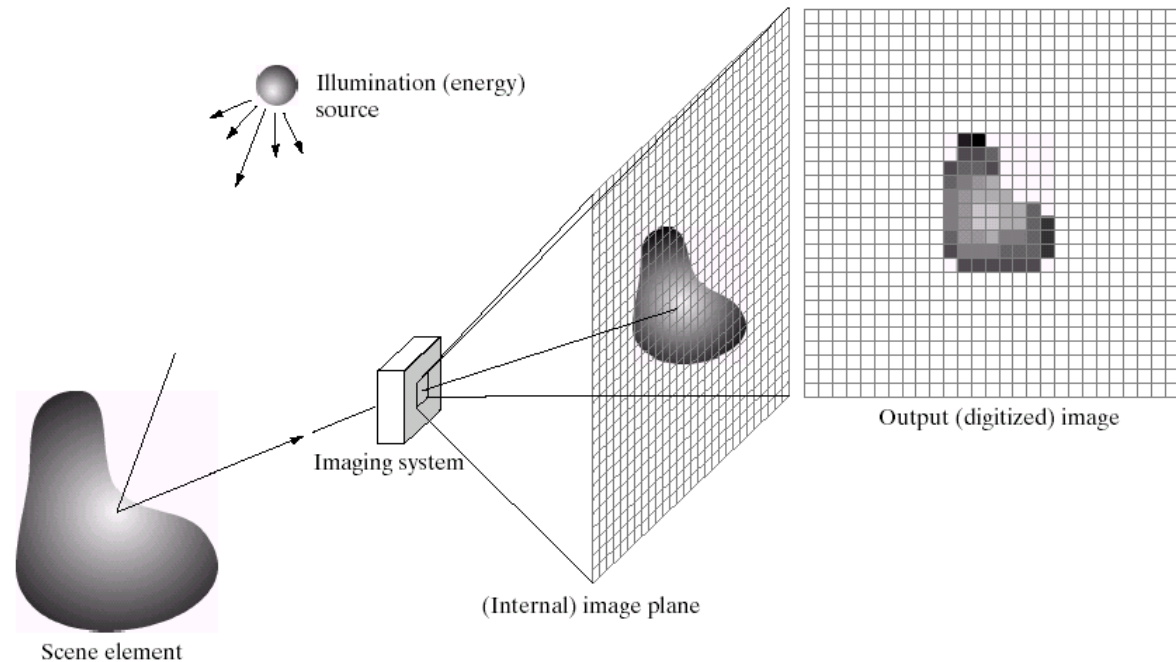


Computer Vision



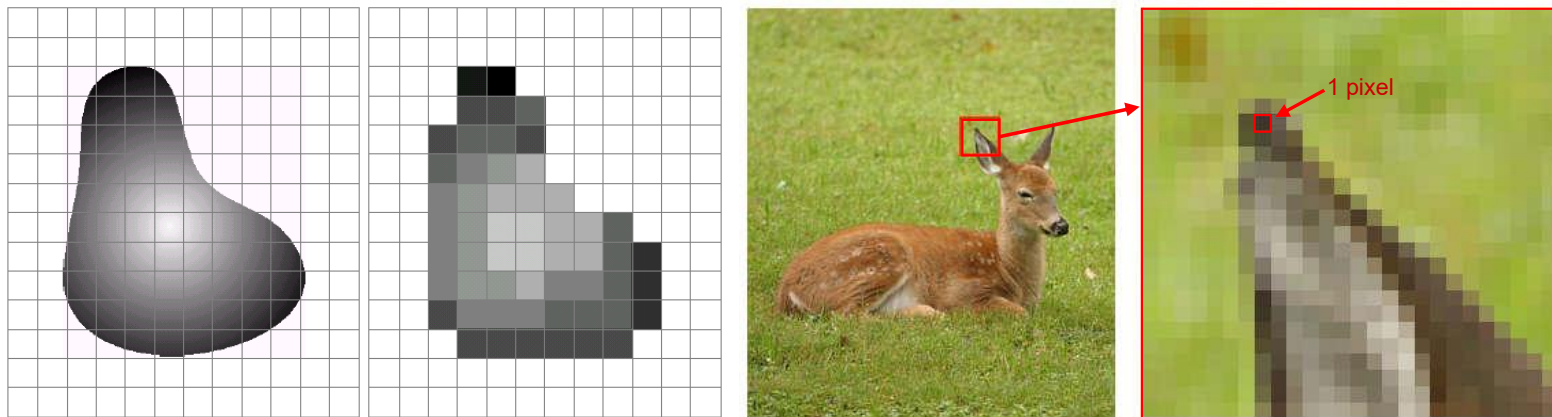
What is a Digital Image?

- A **digital image** is a representation of a two-dimensional image as a finite set of digital values, called picture elements or pixels



What is a Digital Image? (cont...)

- Pixel values typically represent gray levels, colours, heights, opacities etc
- **Remember** *digitization* implies that a digital image is an *approximation* of a real scene



What is a Digital Image? (cont...)

- Common image formats include:
 - 1 sample per point (B&W or Grayscale)
 - 3 samples per point (Red, Green, and Blue)
 - 4 samples per point (Red, Green, Blue, and “Alpha”, a.k.a. Opacity)



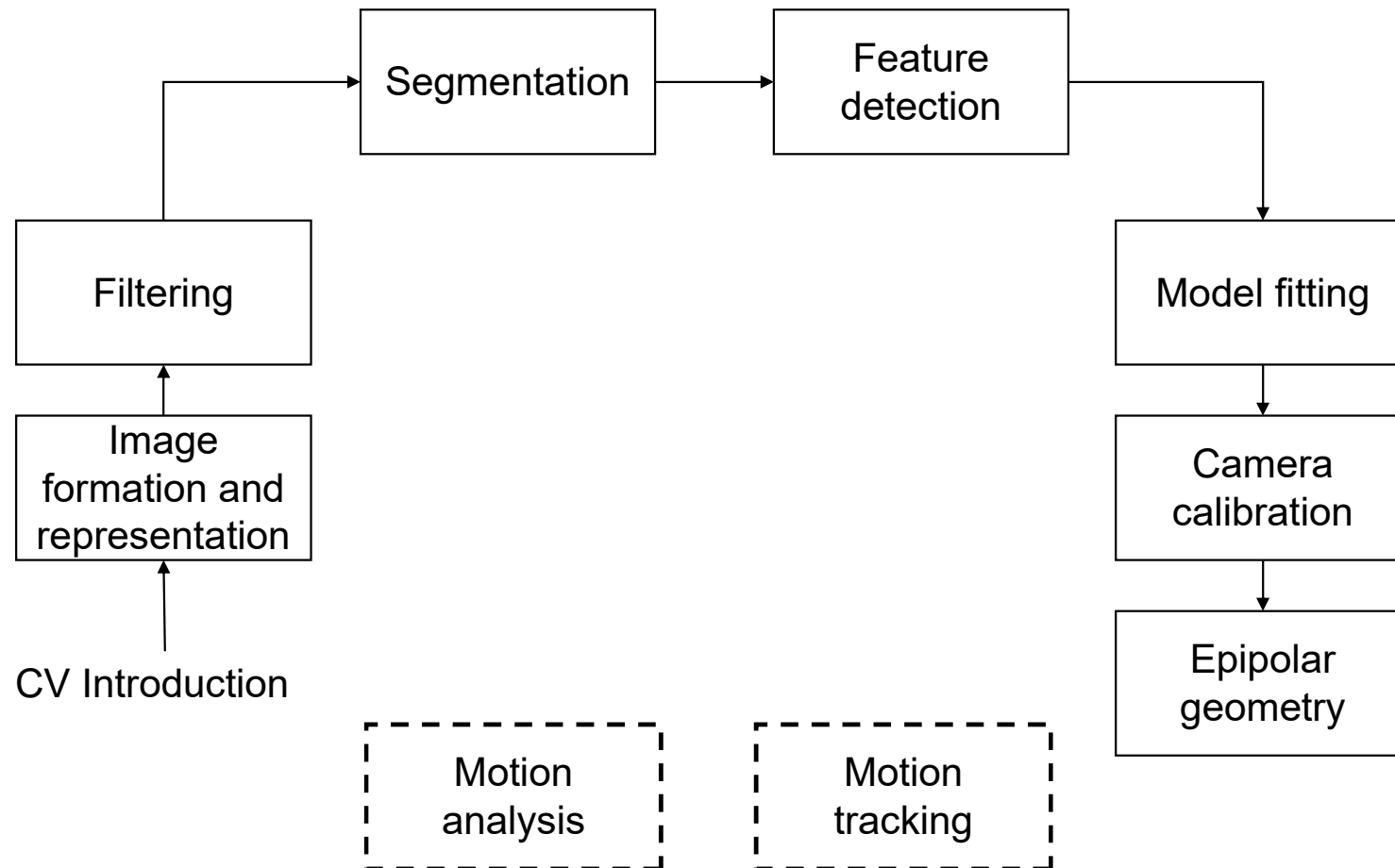
- For most of this course we will focus on grey-scale images

How computer sees an image



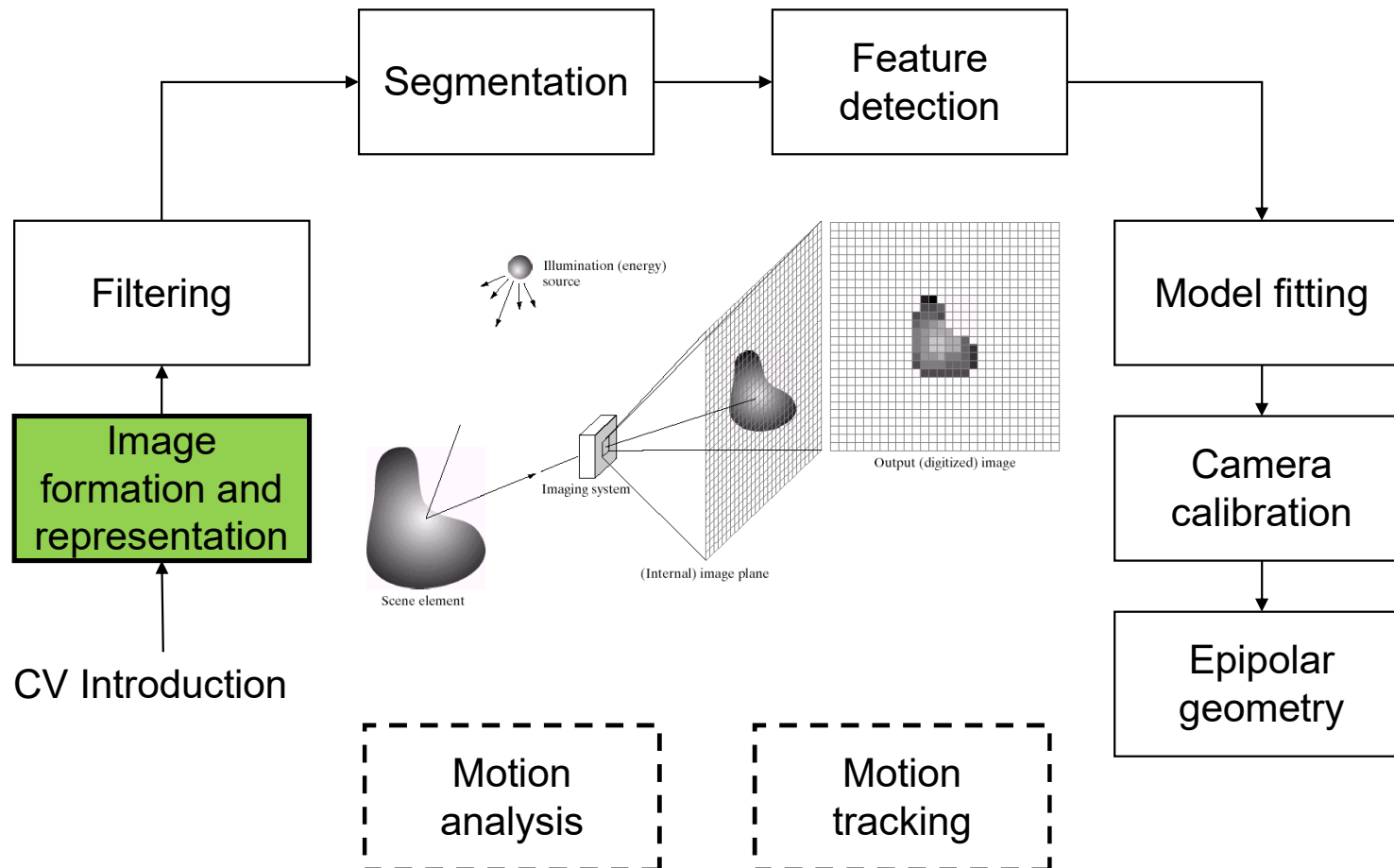
0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

Contents

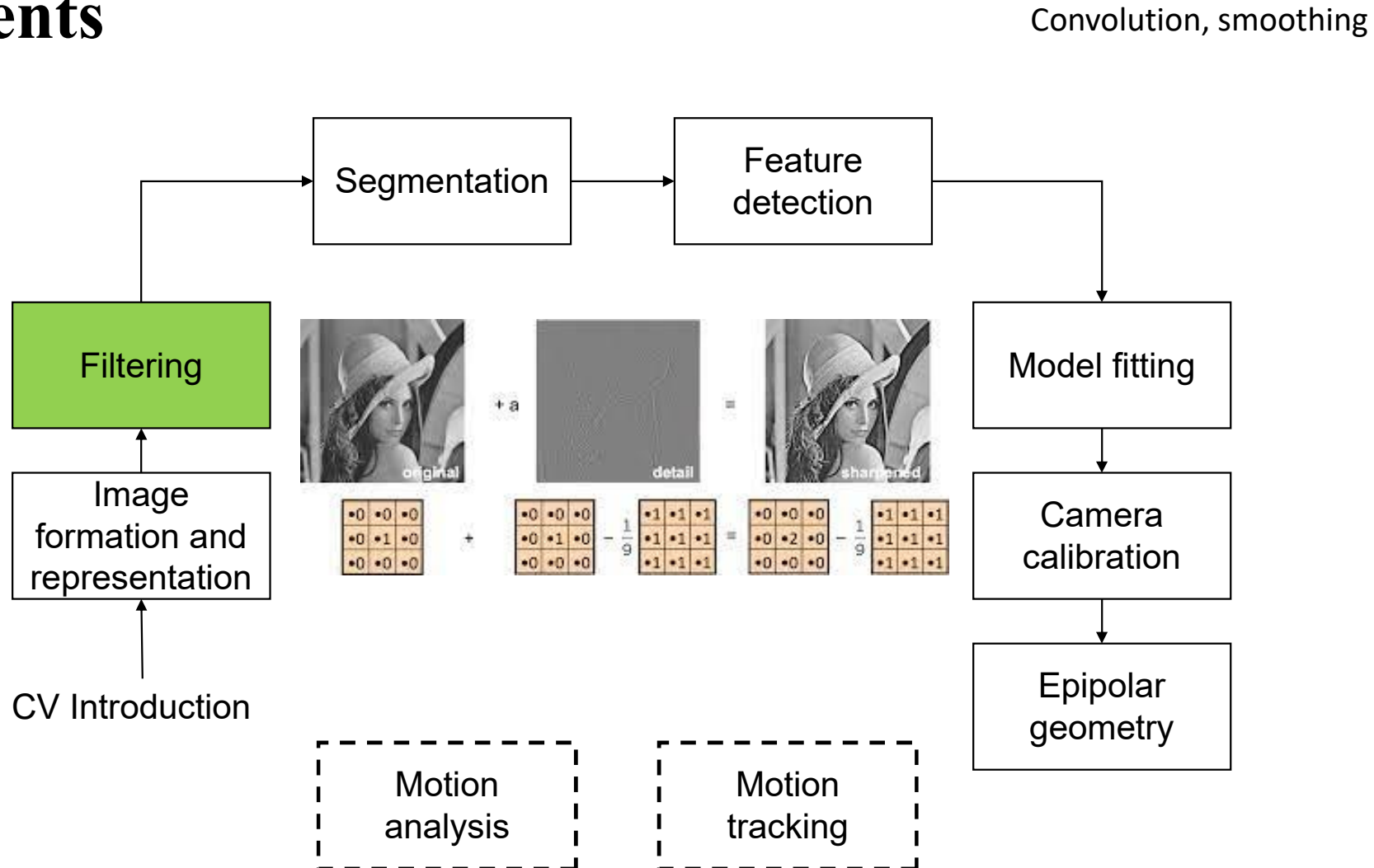


Contents

Imaging geometry, digitization, cameras and projections, rigid and affine transformations

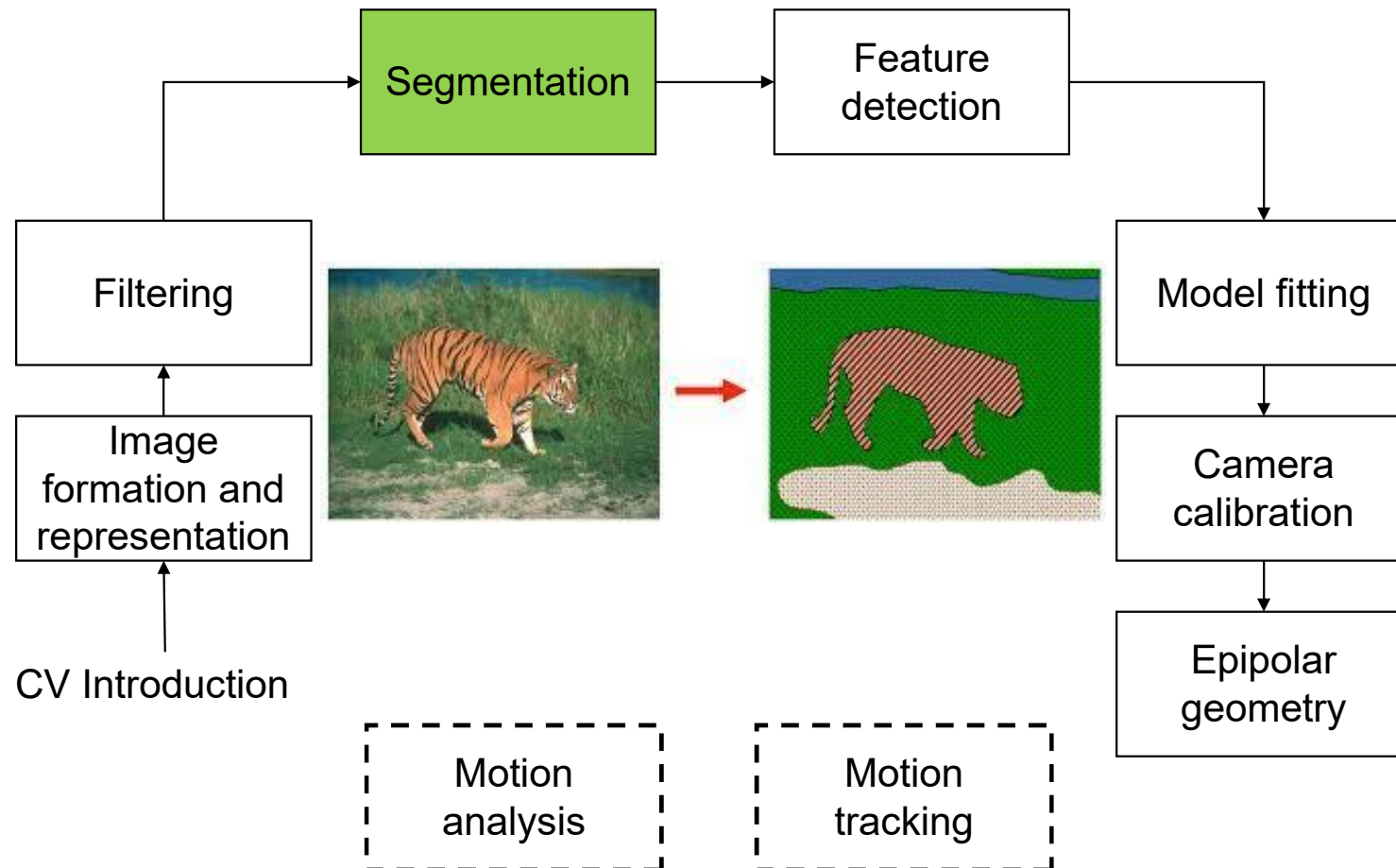


Contents



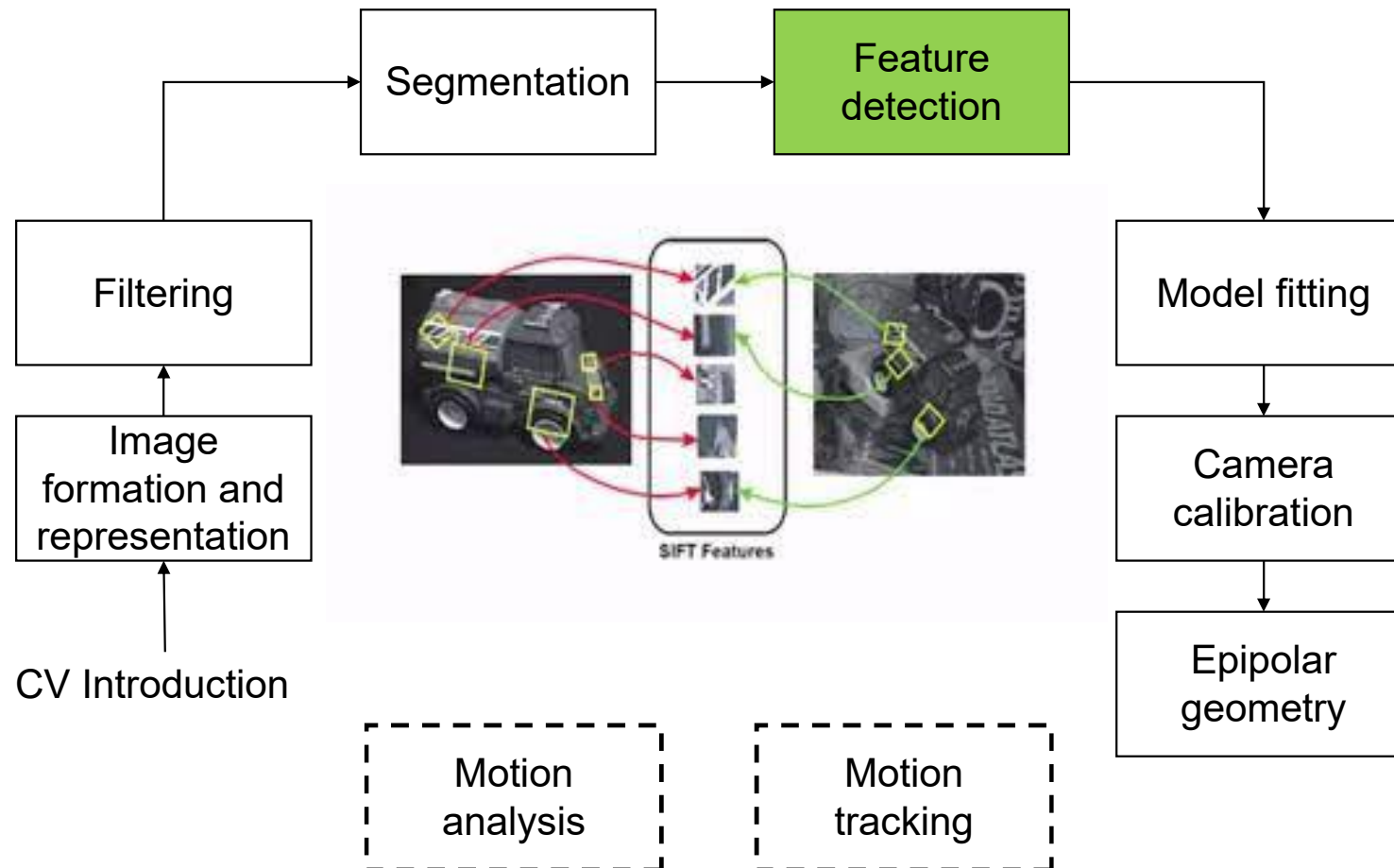
Contents

Region splitting and merging; quadtree structures for segmentation



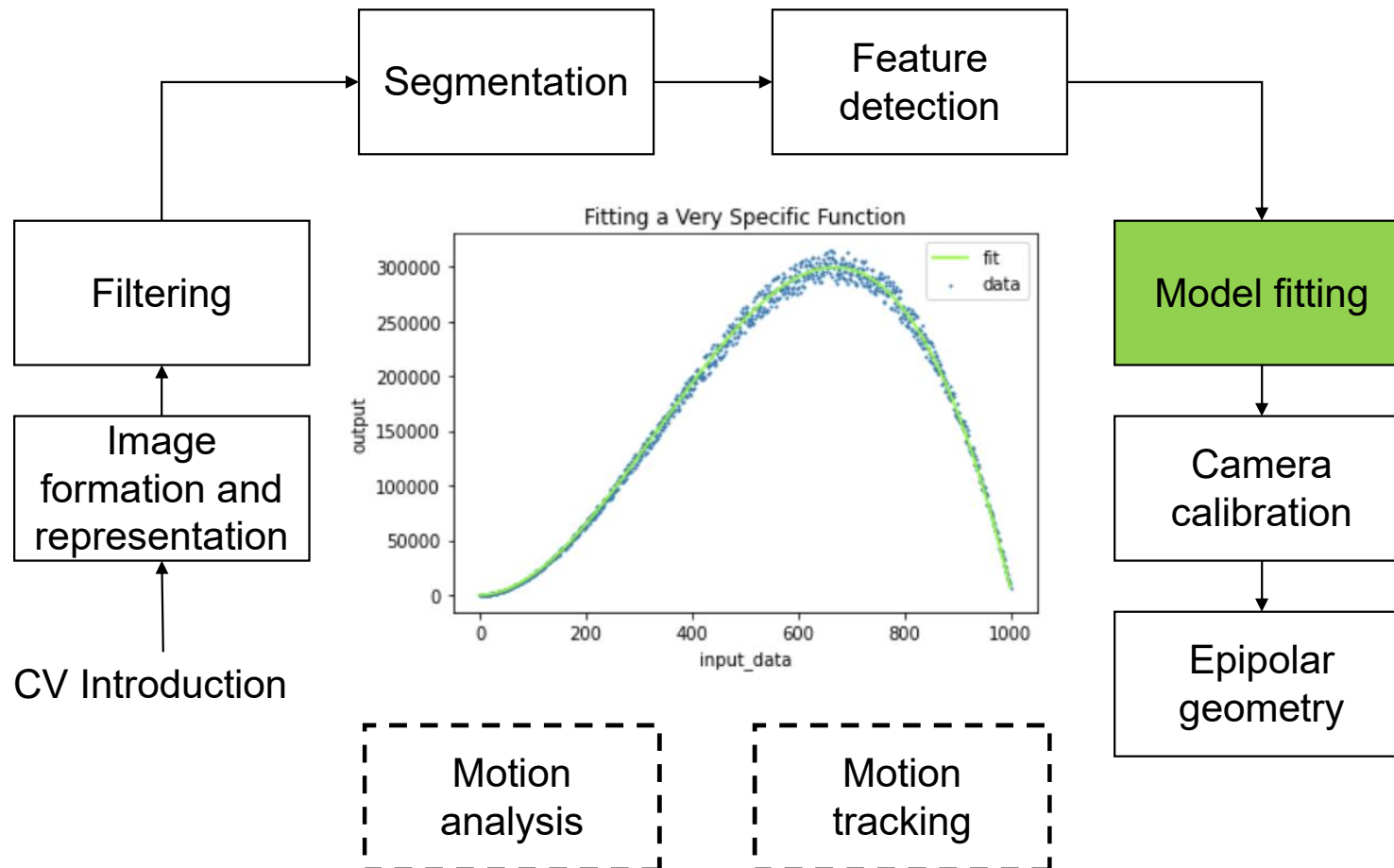
Contents

Edge detection, corner detection, line and curve detection, SIFT and HOG descriptors, shape context descriptors



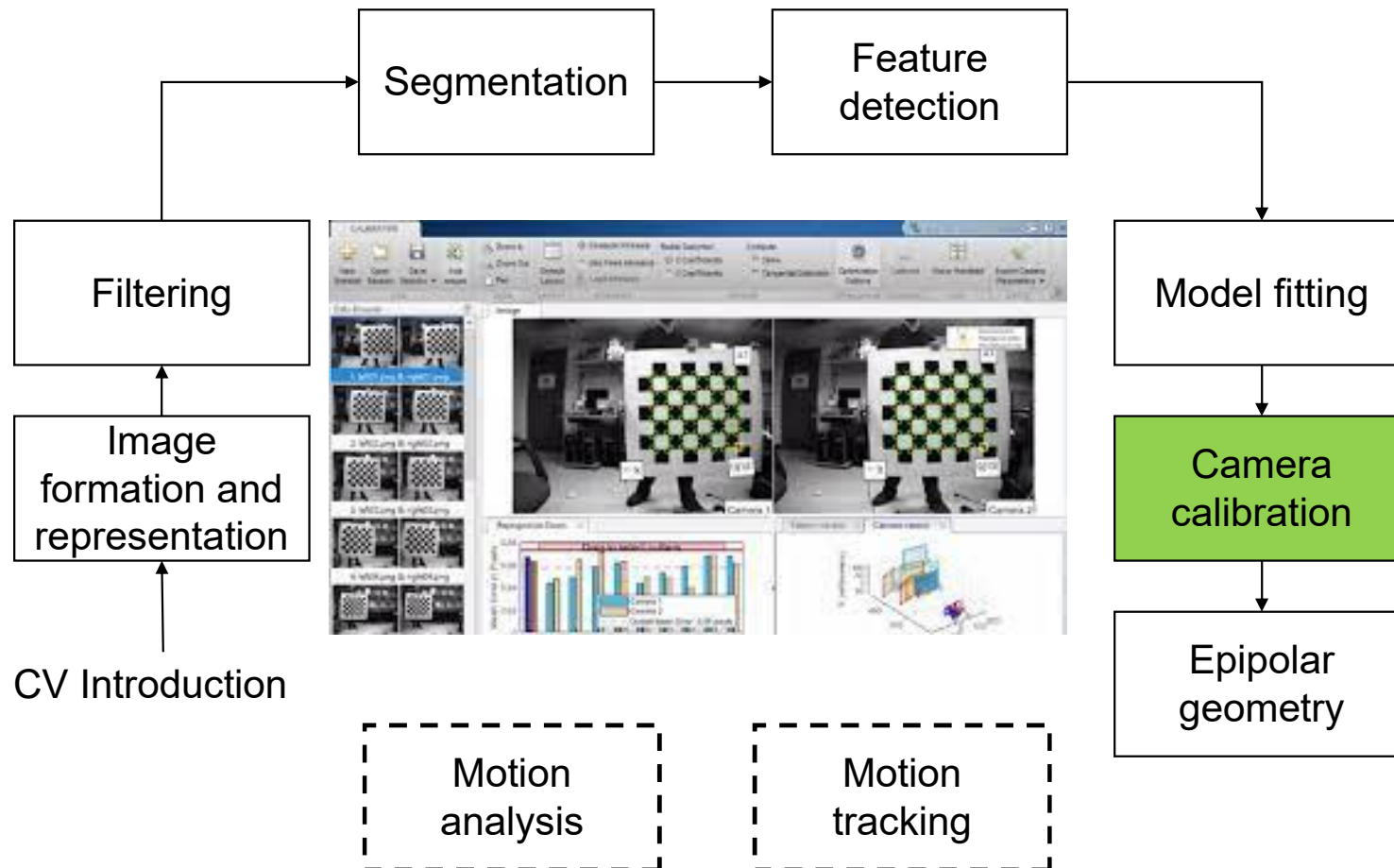
Contents

Hough transform, line fitting, ellipse and conic sections fitting, algebraic and Euclidean distance measures



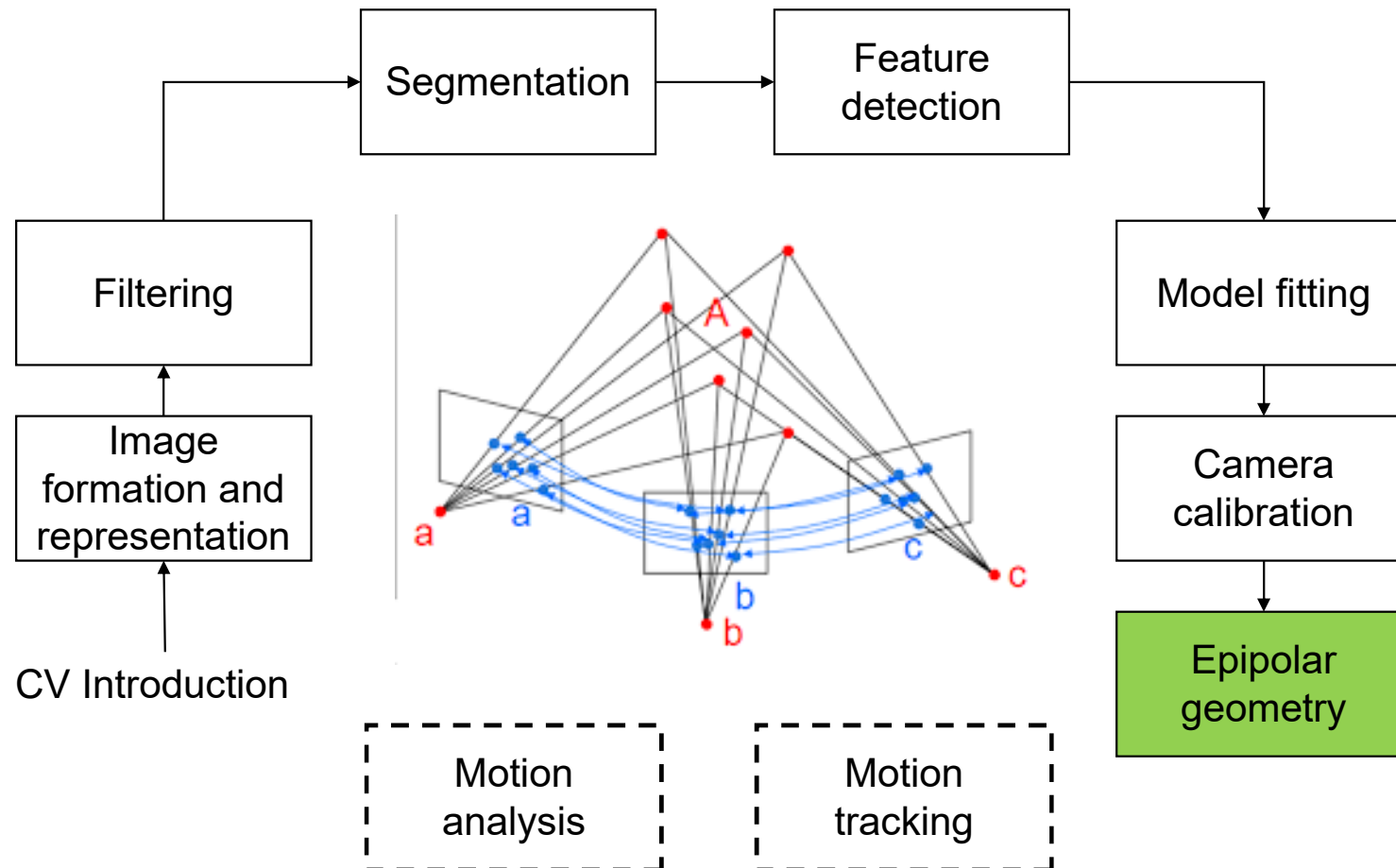
Contents

Camera models; intrinsic and extrinsic parameters; affine, and perspective camera models



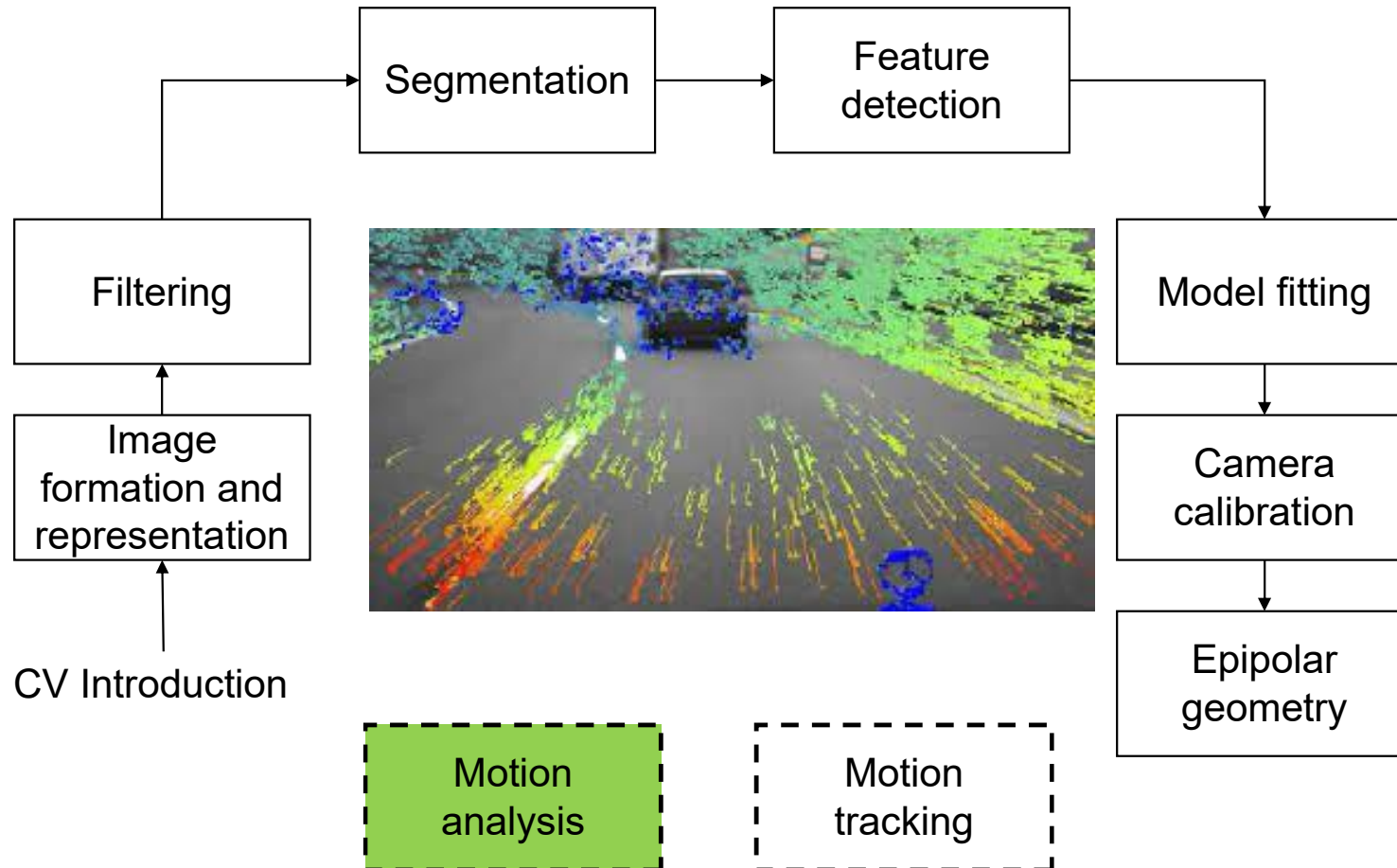
Contents

Introduction to projective geometry; epipolar constraints; the essential and fundamental matrices



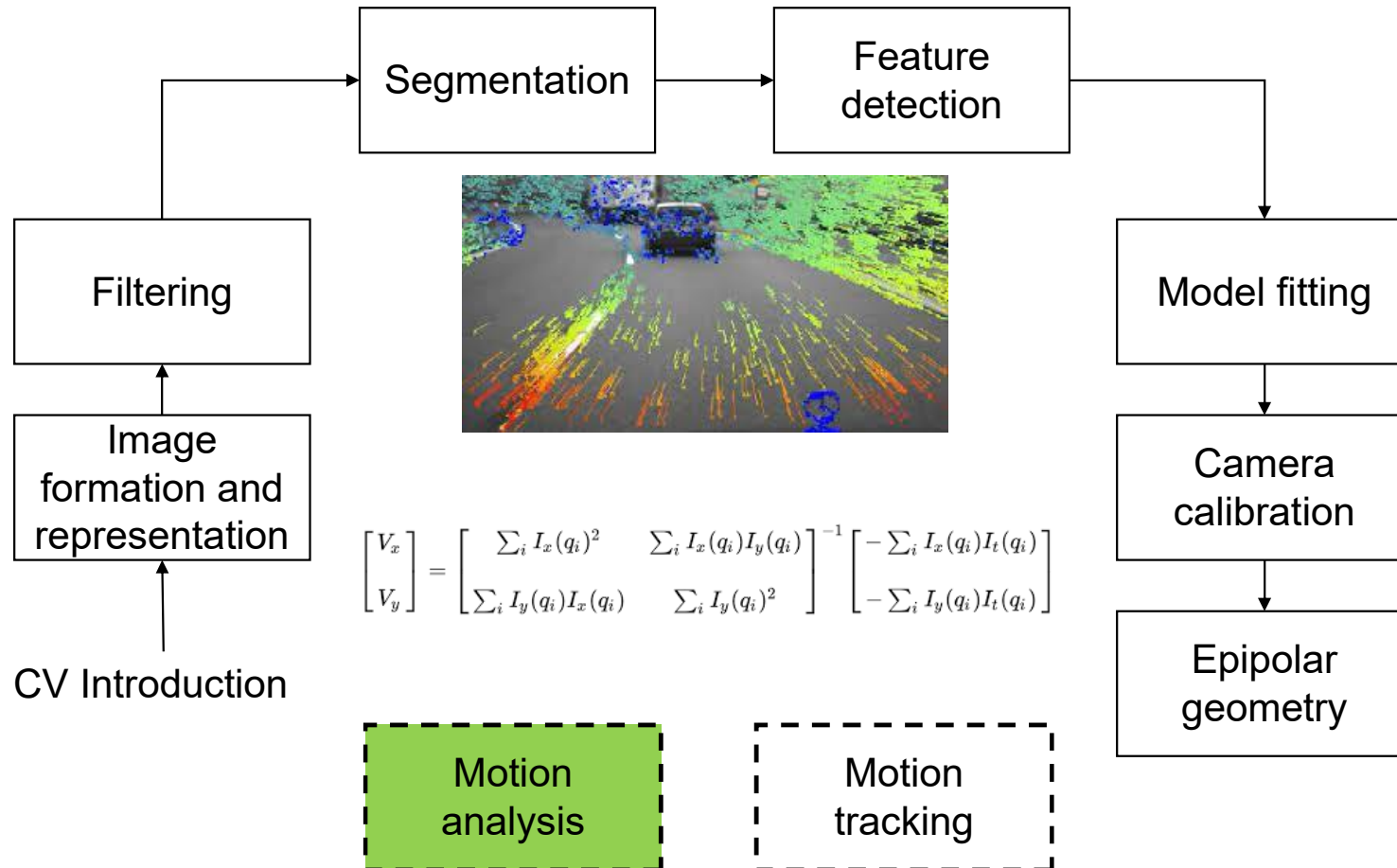
Contents

The motion field of rigid objects; motion parallax; optical flow, the image brightness constancy equation, affine flow; differential techniques; feature-based techniques



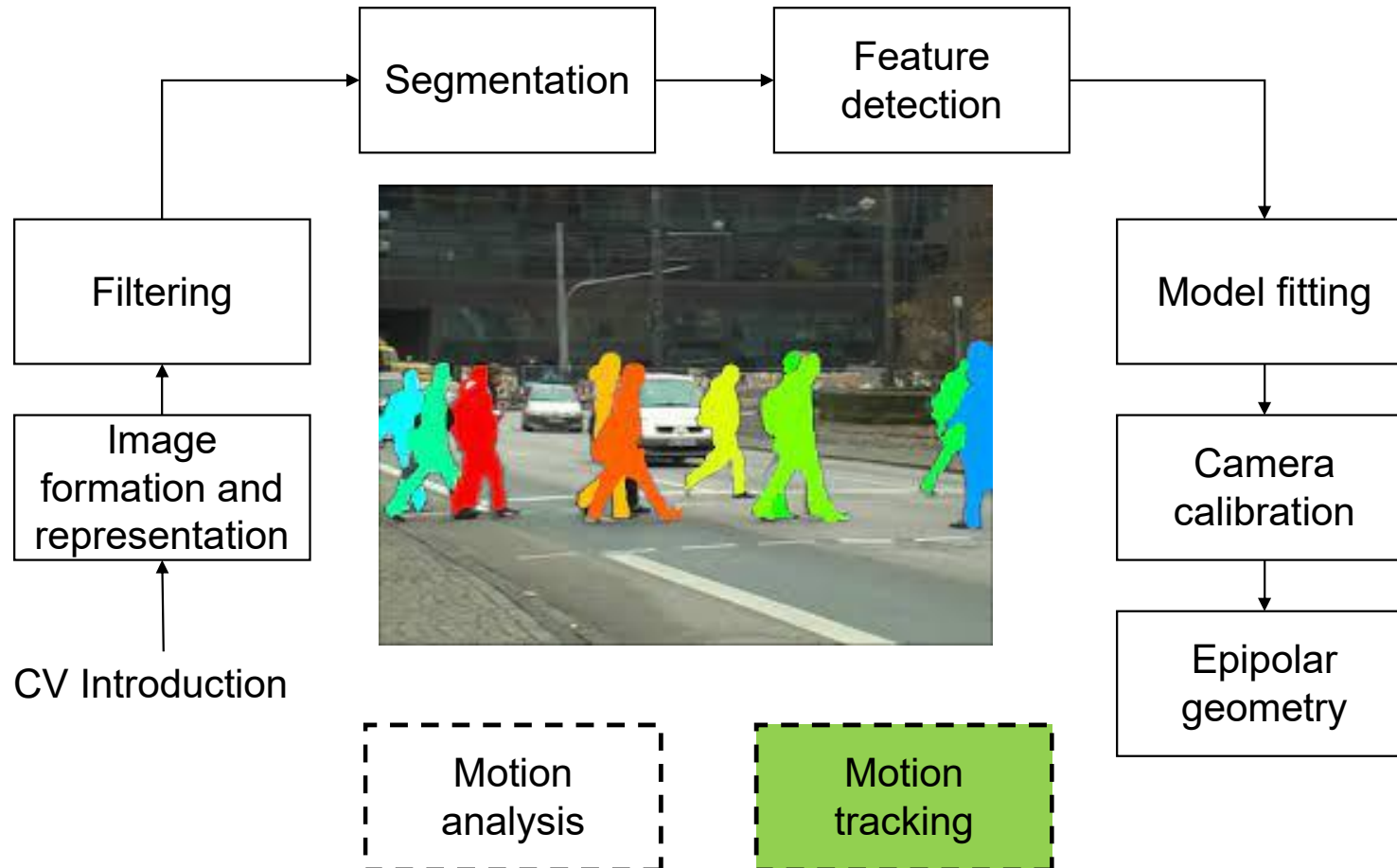
Contents

The motion field of rigid objects; motion parallax; optical flow, the image brightness constancy equation, affine flow; differential techniques; feature-based techniques



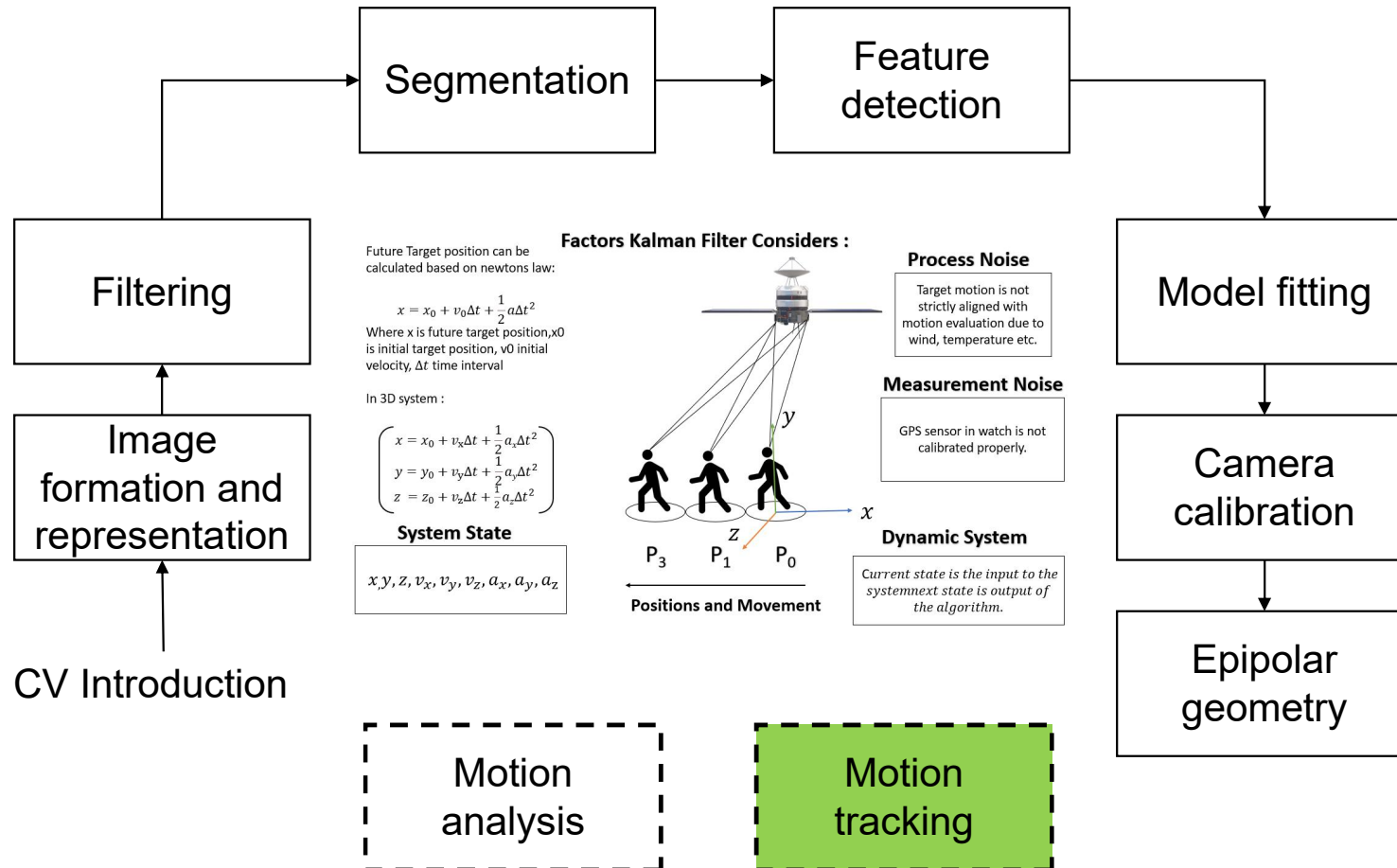
Contents

The kalman filter; object recognition and shape representation.

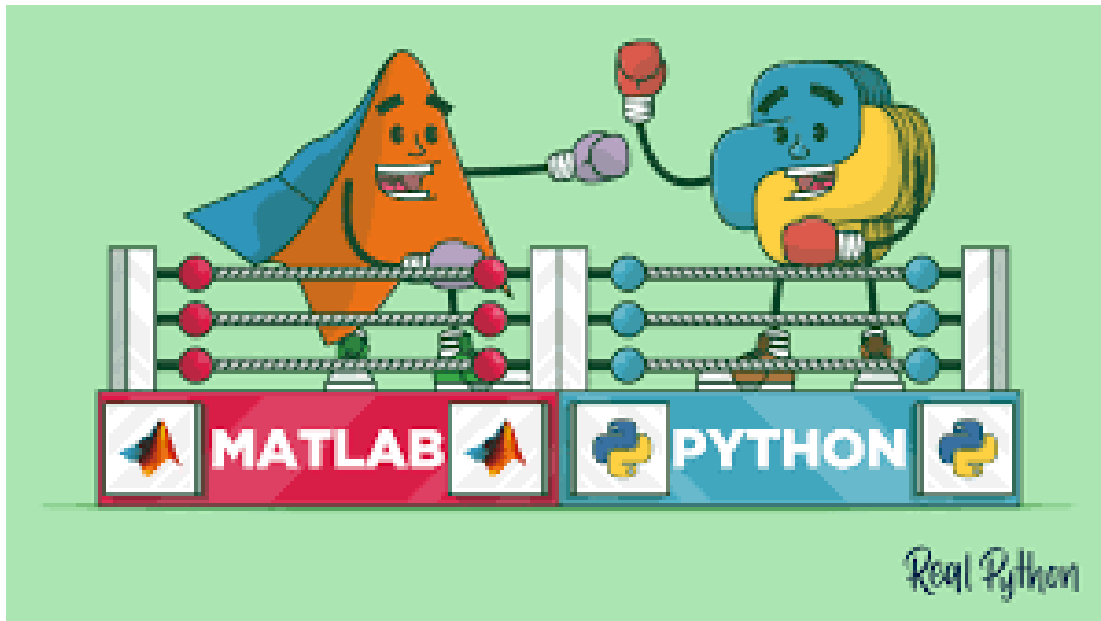


Contents

The kalman filter; object recognition and shape representation.



Matlab Vs Python

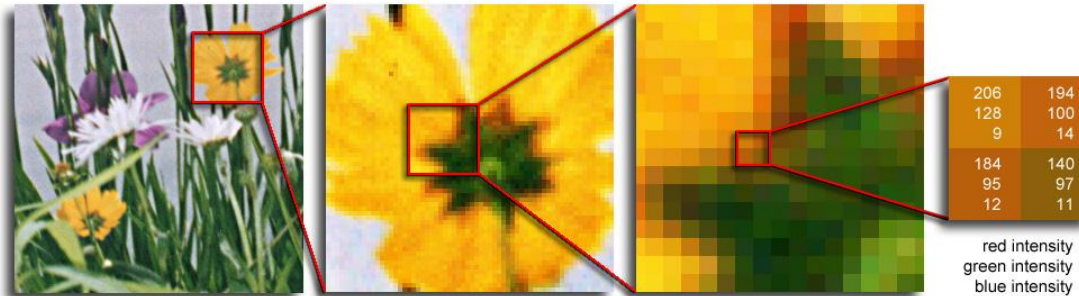


- Quiz
- Assignments
- Midterm
- Final

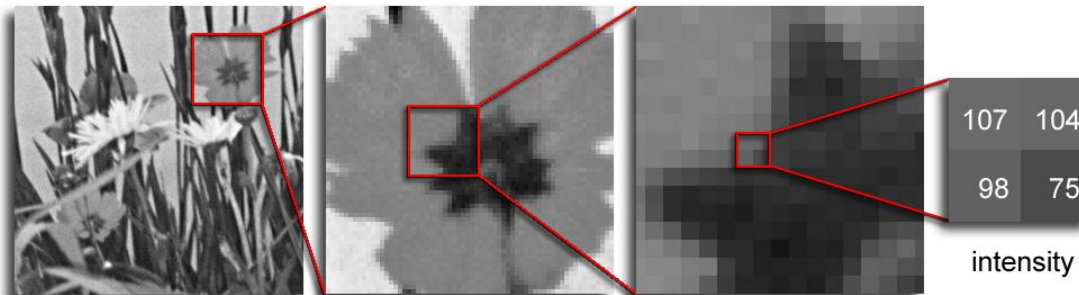
Digital Image

Color images have 3 values per pixel; monochrome images have 1 value per pixel.

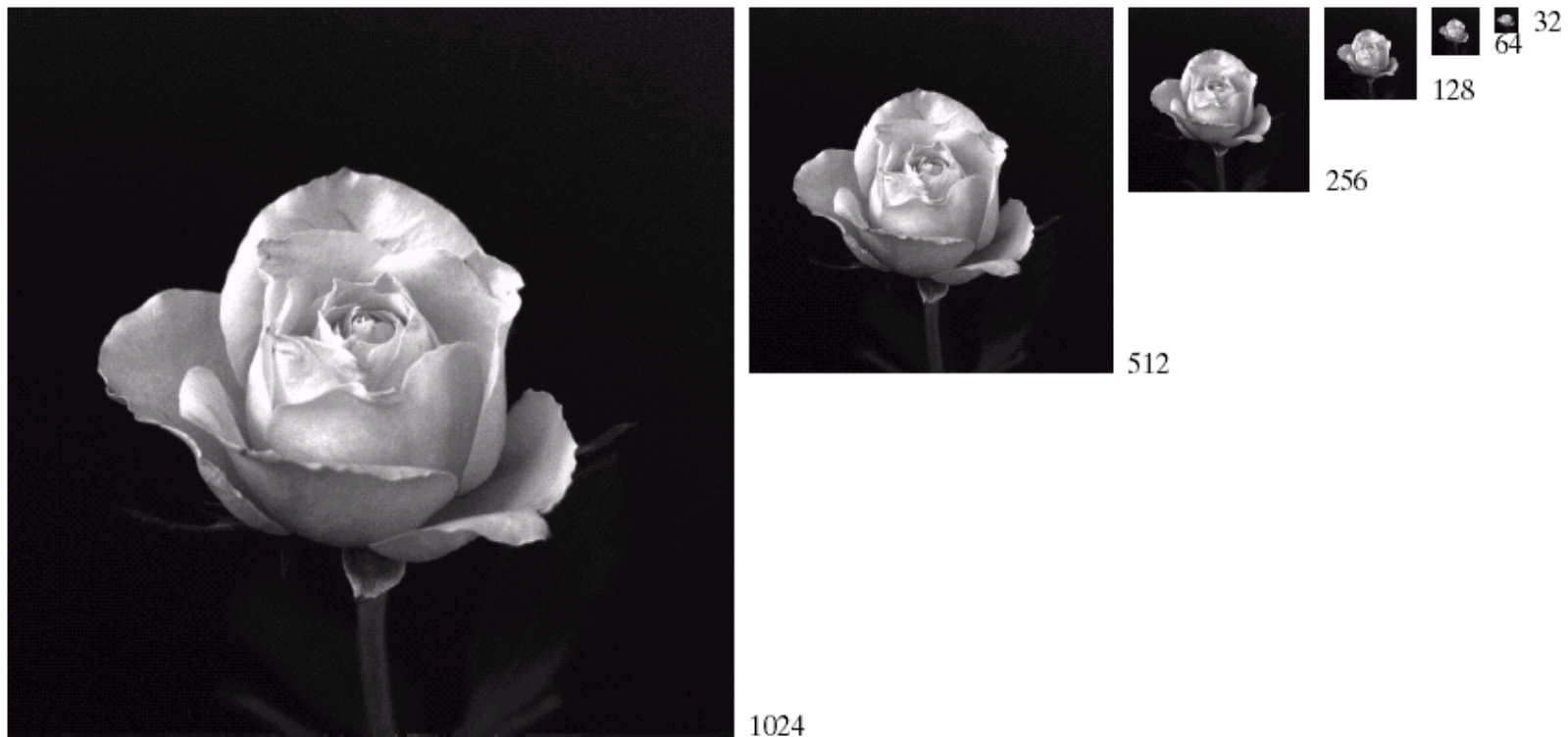
a grid of squares,
each of which
contains a single
color



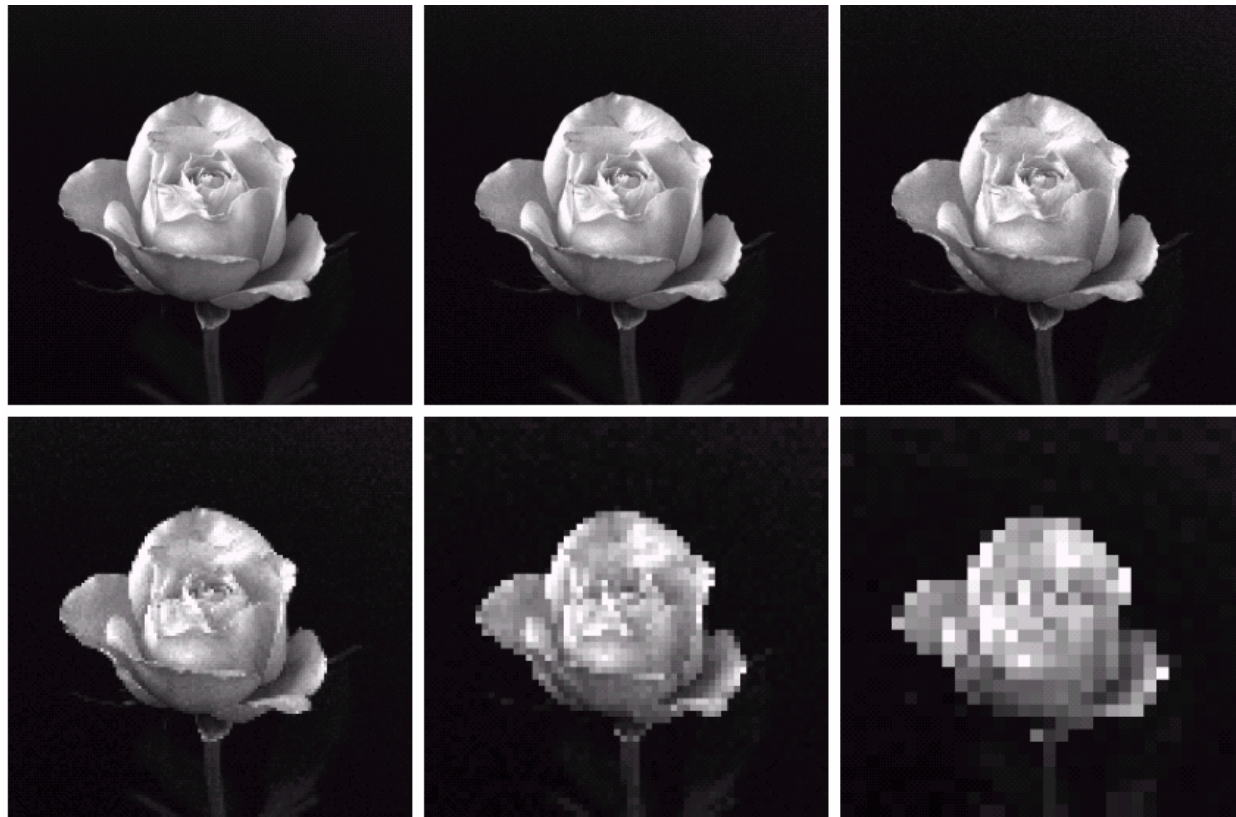
each square is
called a pixel (for
picture element)



Spatial Resolution



Spatial Resolution



Resolution: How much is enough?

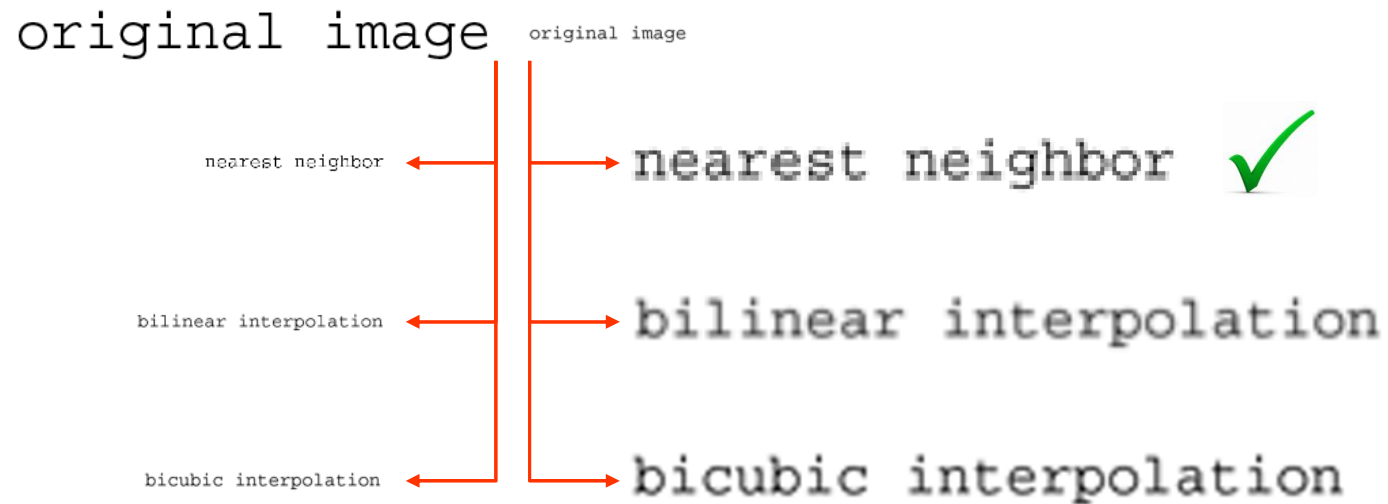
- ◆ Depends on what is in the image and what you would like to do with it



The picture on the right is fine for counting the number of cars, but not for reading the number plate

Image Interpolation

- ◆ Image resizing
- ◆ Three methods



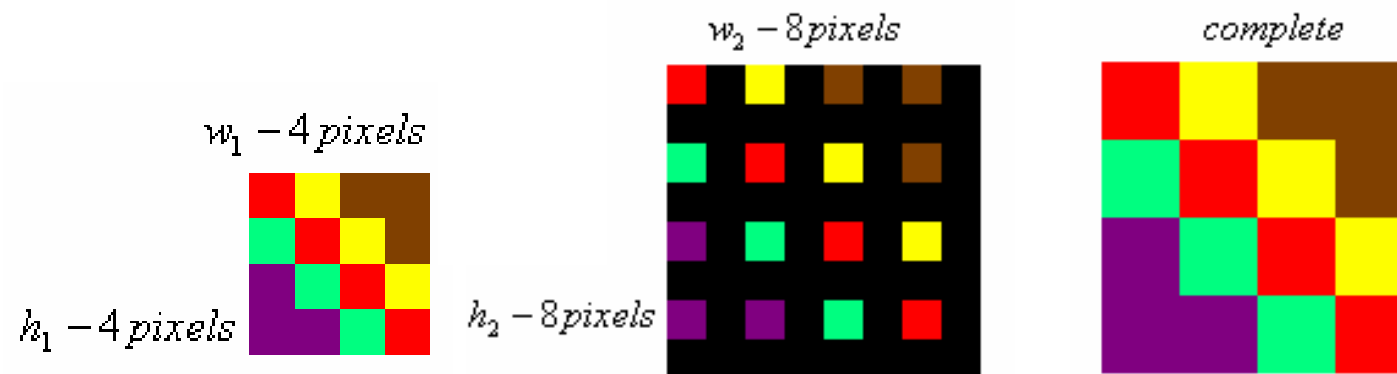
Enlarging an Image

- ◆ Pixel replication

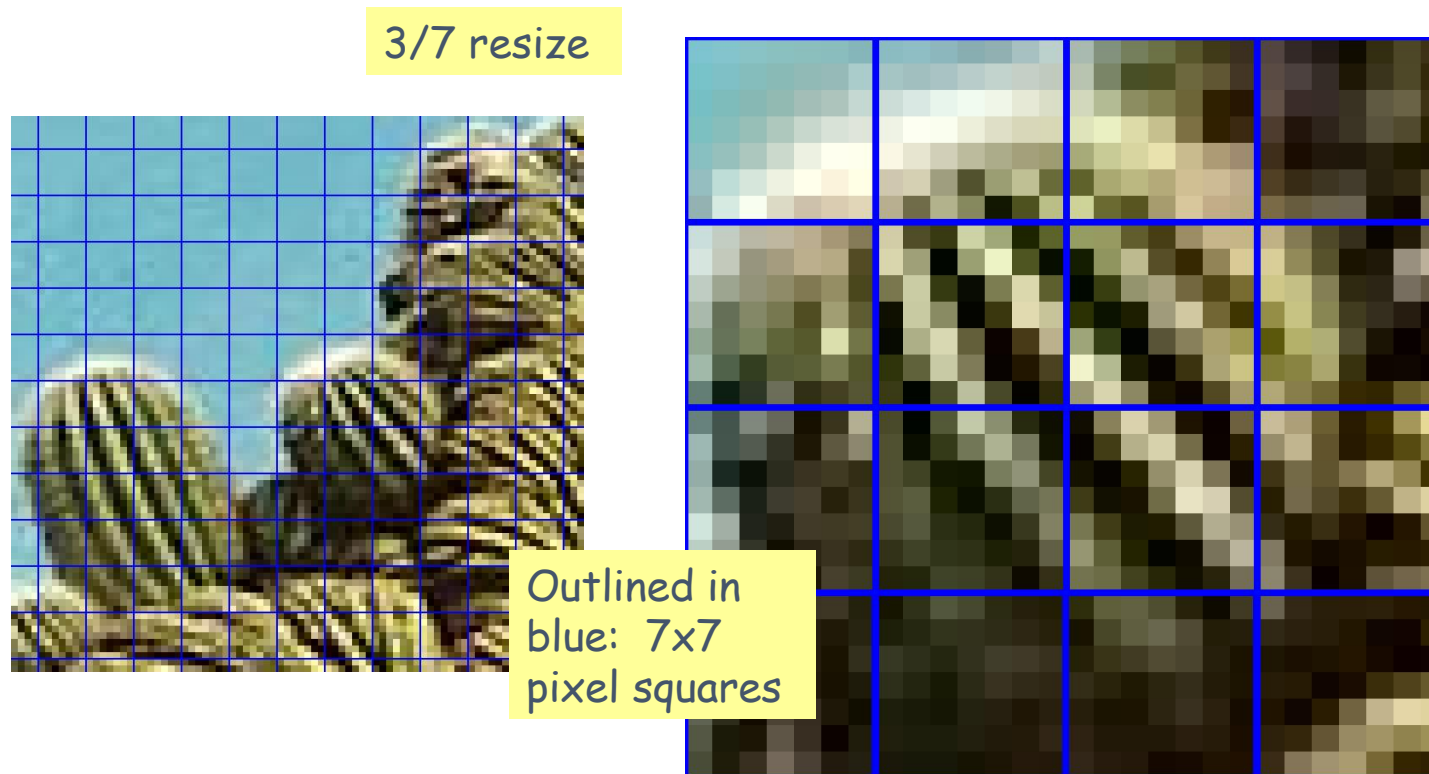
[1 2 3 4 5]

[1 1 2 2 3 3 4 4 5 5] (One step)

[1 1 1 2 2 2 3 3 3 4 4 4 5 5 5] (Two step)



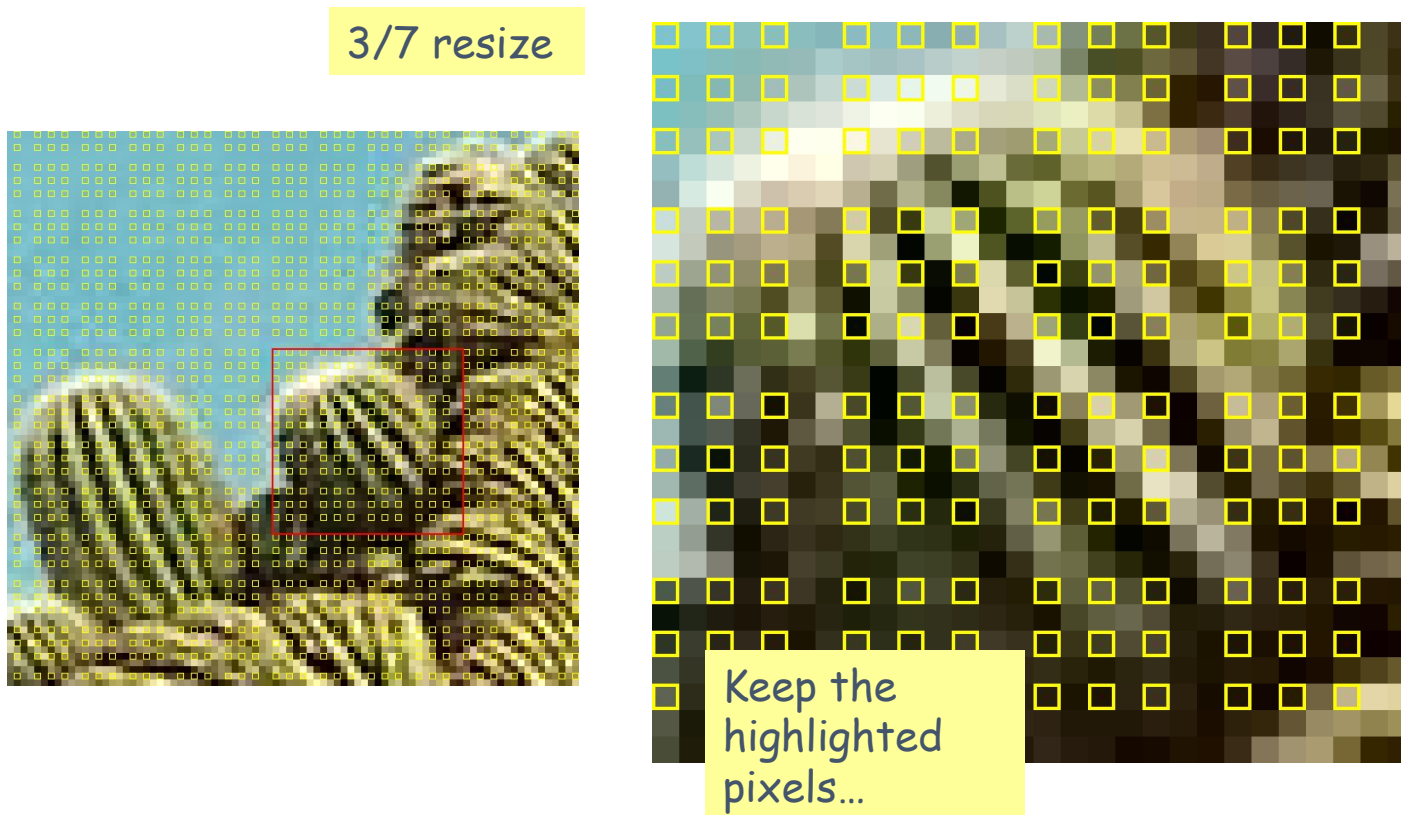
Nearest Neighbor Interpolation



Nearest Neighbor Interpolation

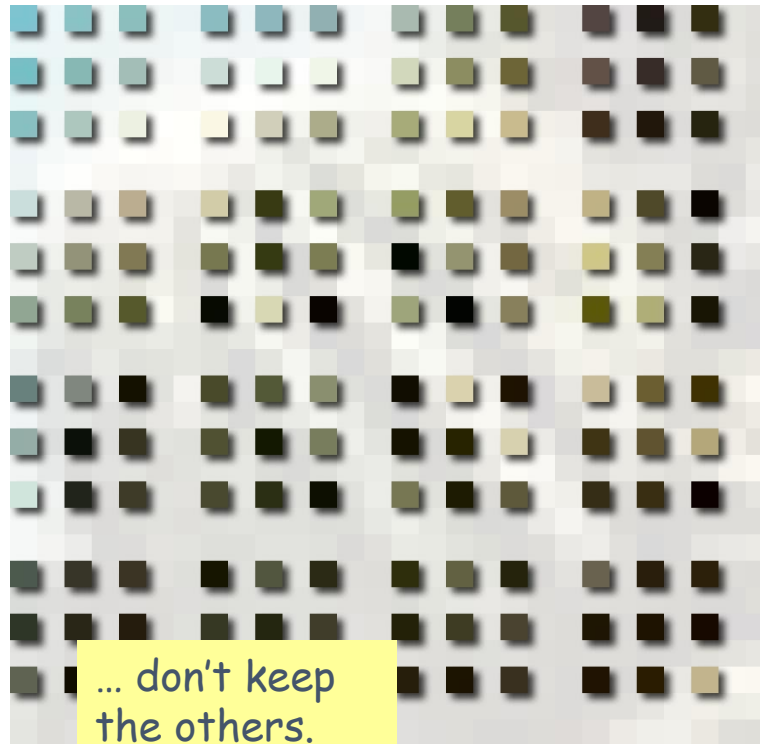
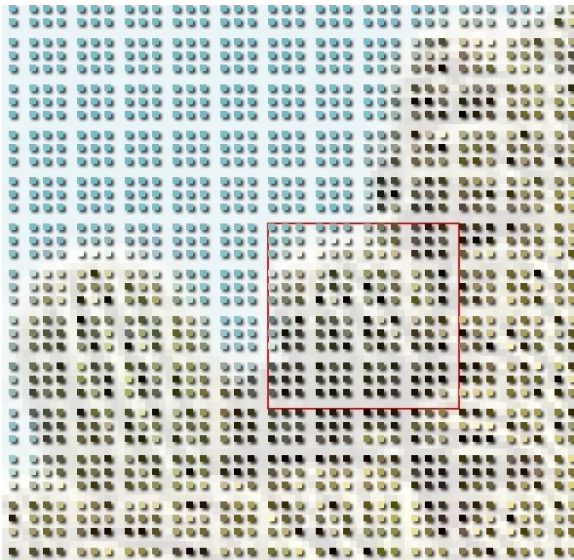


Nearest Neighbor Interpolation

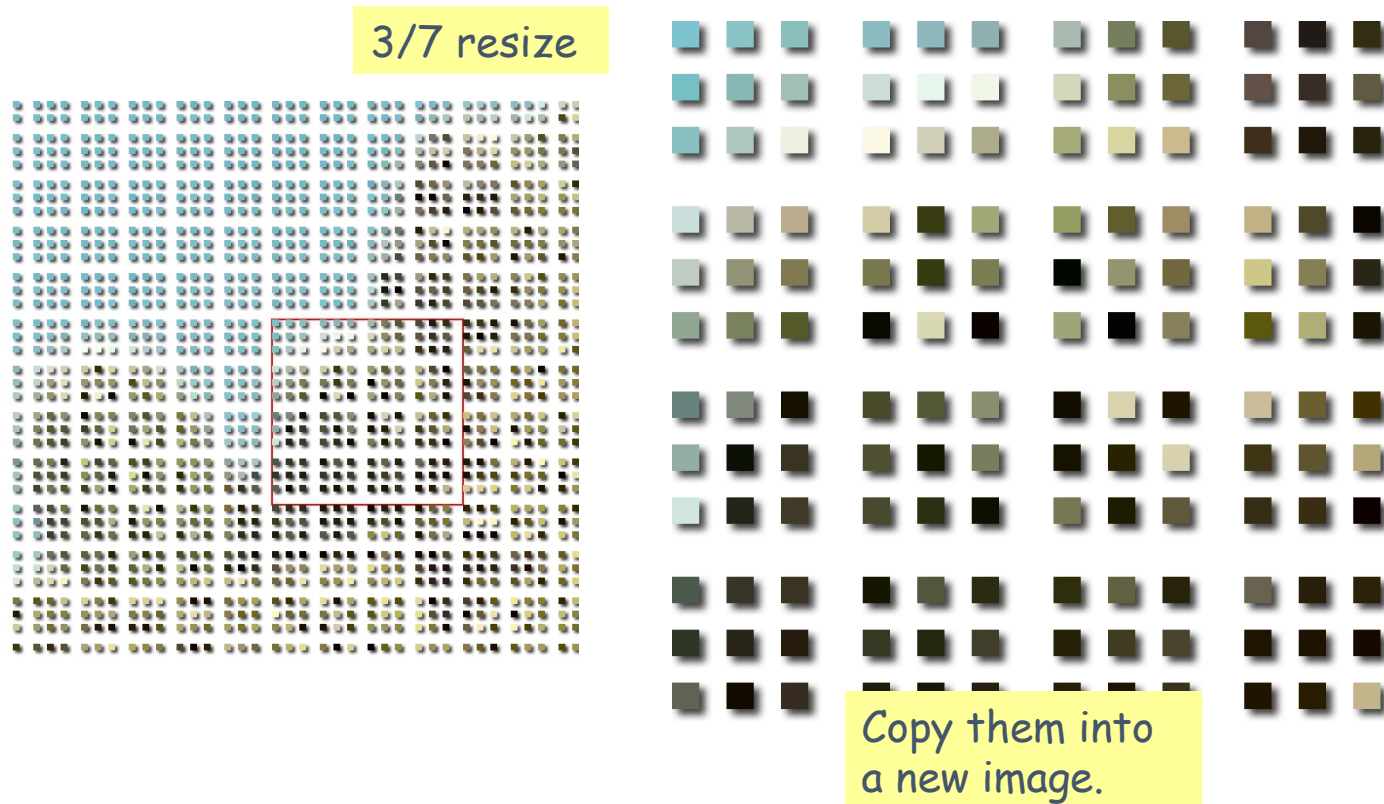


Nearest Neighbor Interpolation

3/7 resize

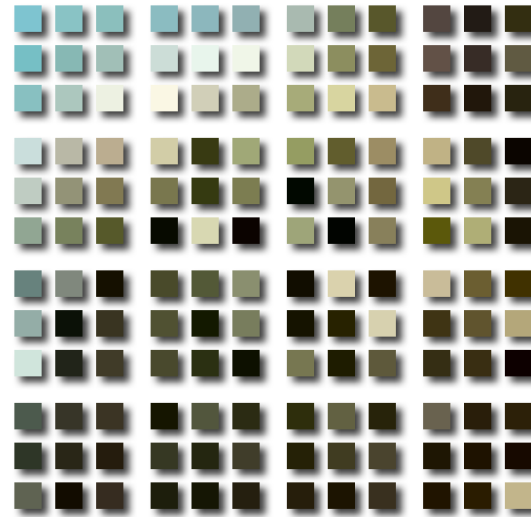


Nearest Neighbor Interpolation



Nearest Neighbor Interpolation

3/7 resize



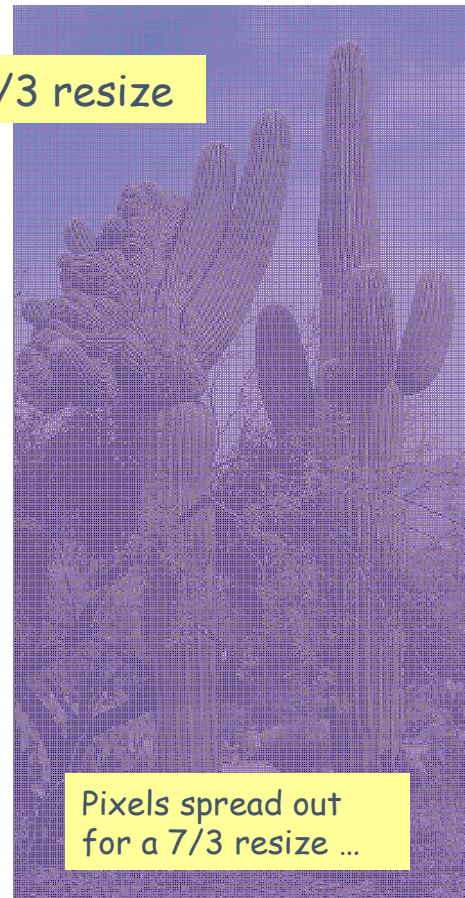
Copy them into
a new image.

Nearest Neighbor Interpolation



Original
image

$7/3$ resize

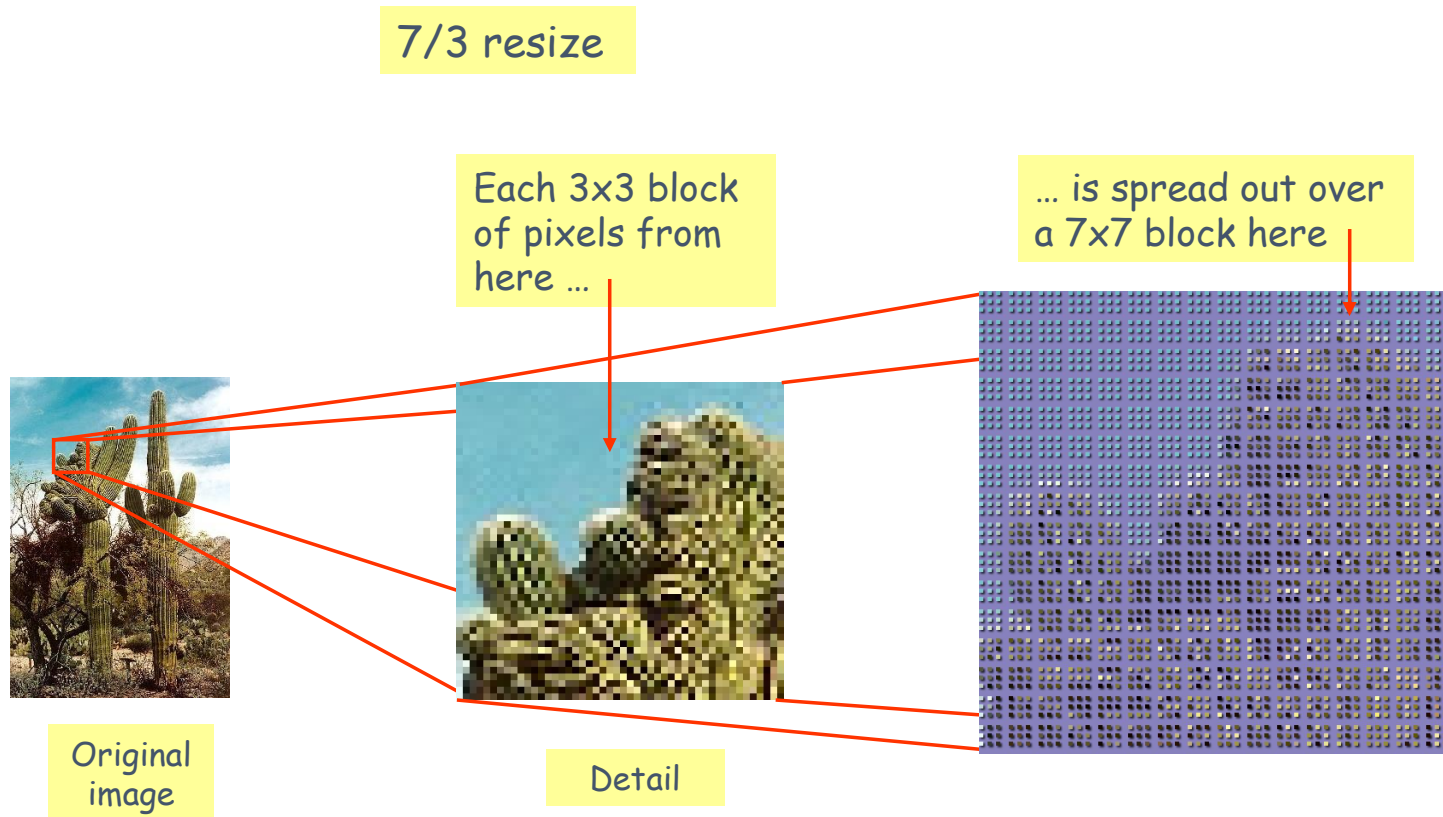


Pixels spread out
for a $7/3$ resize ...



... then filled in.

Nearest Neighbor Interpolation

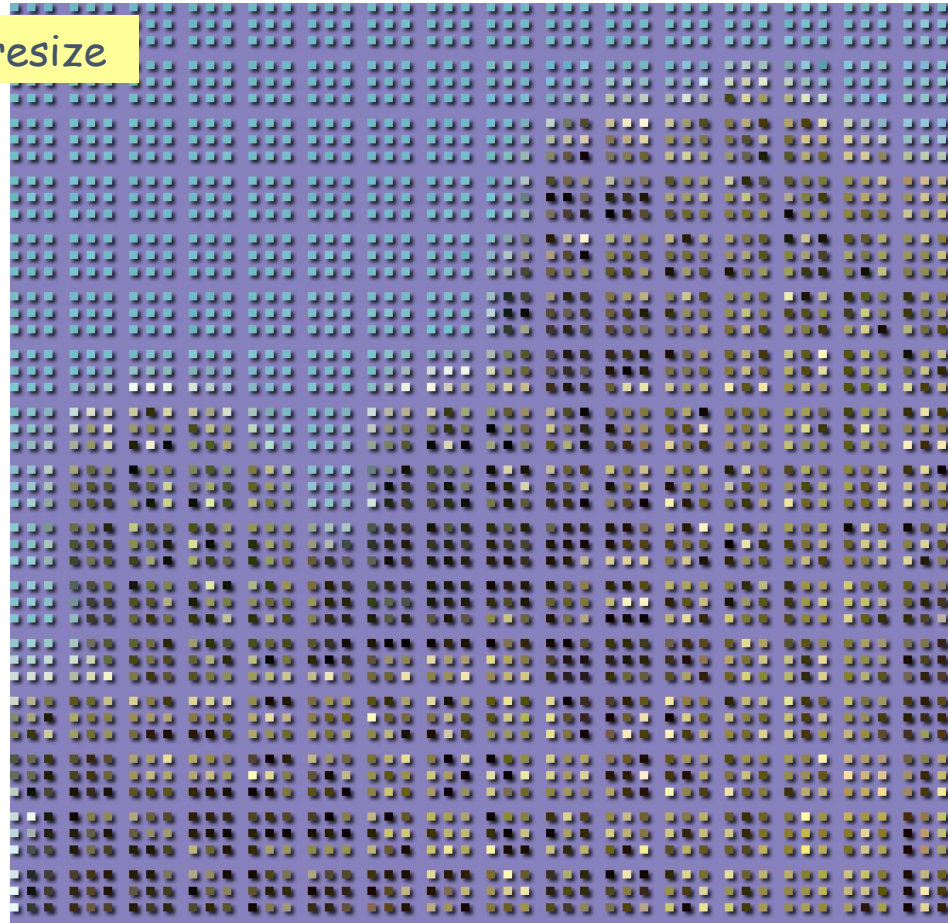


Nearest Neighbor Interpolation



3x3 blocks
distributed over
7x7 blocks

7/3 resize

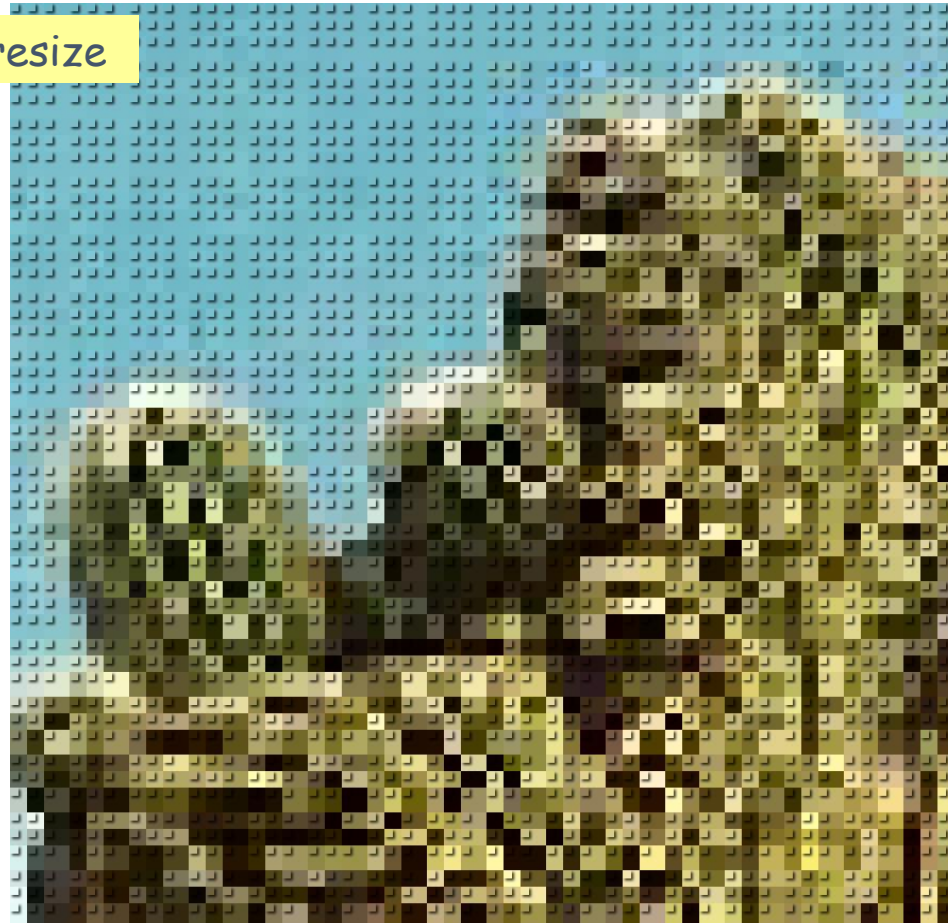


Nearest Neighbor Interpolation



Empty pixels filled
with color from non-
empty pixel

7/3 resize

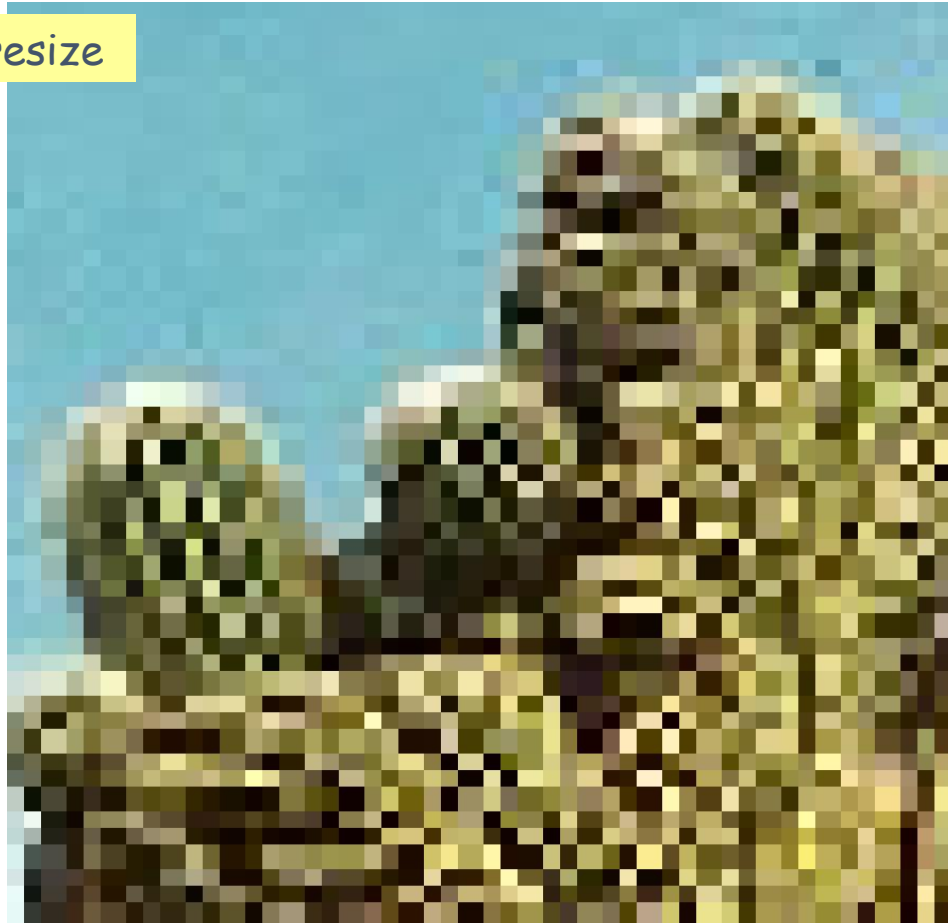


Nearest Neighbor Interpolation

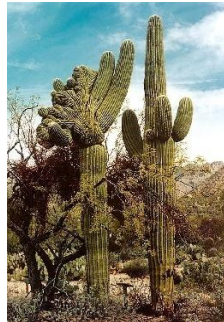


Empty pixels filled
with color from non-
empty pixel

7/3 resize



Nearest Neighbor Interpolation



Original
image



7/3 resized

Image Interpolation

- ◆ Nearest neighbour interpolation
 - Simple but produces undesired artefacts
- ◆ Bilinear Interpolation
 - Contribution from 4 neighbors
- ◆ Bicubic Interpolation
 - Contribution from 16 neighbors

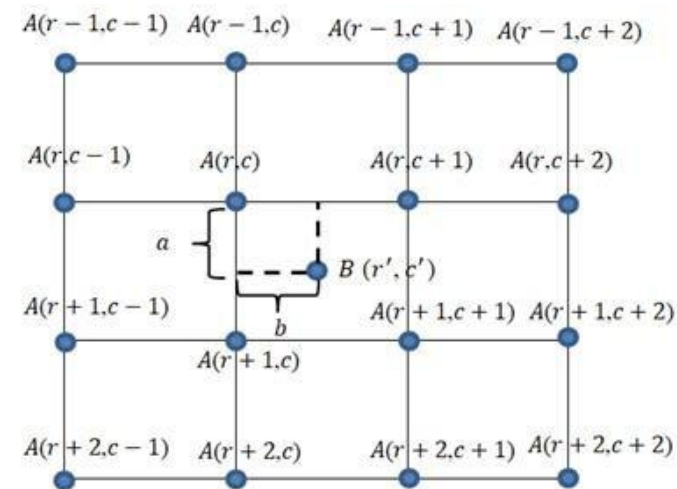
Image Interpolation

- ◆ Nearest neighbour interpolation
- ◆ Bilinear Interpolation

$$f(x, y) = \frac{(x_2 - x)(y_2 - y)}{(x_2 - x_1)(y_2 - y_1)} Q_{11} + \frac{(x - x_1)(y_2 - y)}{(x_2 - x_1)(y_2 - y_1)} Q_{21} + \frac{(x_2 - x)(y - y_1)}{(x_2 - x_1)(y_2 - y_1)} Q_{12} + \frac{(x - x_1)(y - y_1)}{(x_2 - x_1)(y_2 - y_1)} Q_{22}$$

- ◆ Bicubic Interpolation

$$p(x, y) = \sum_{i=0}^3 \sum_{j=0}^3 a_{ij} x^i y^j.$$



- Install Anaconda* * any version
- Install
 - python == 3.7.16
 - tensorflow == 2.1.0
 - keras-gpu == 2.3.1
- Install
 - spyder*
- Install
 - From environment opencv == 4.6.0
(import cv2ImportError: DLL load failed: The specified module could not be found.)
 - pip install opencv-python
- Install
 - matplotlib* from environment, matplotlib = 3.5.3